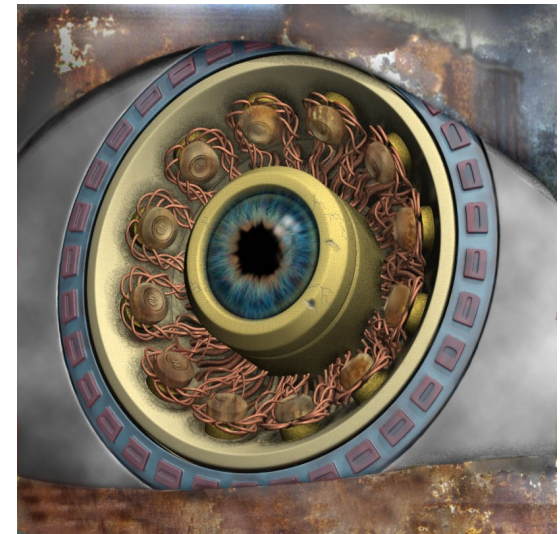
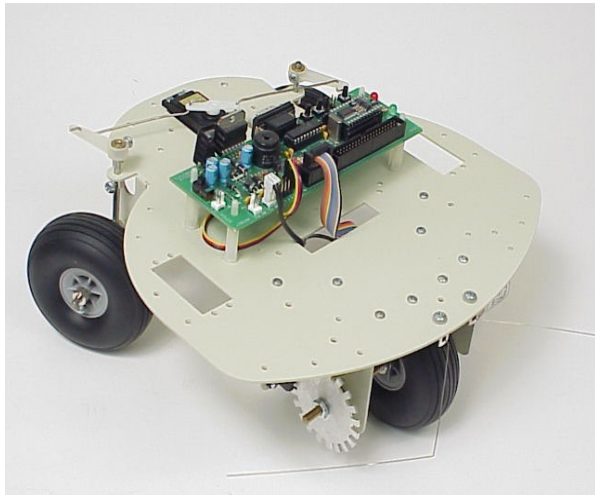




Procesamiento y Análisis de Imágenes Digitales

Robótica y Percepción Computacional

4º Curso. Graduado en Ingeniería Informática

Luis Baumela
Departamento de Inteligencia Artificial
Universidad Politécnica de Madrid



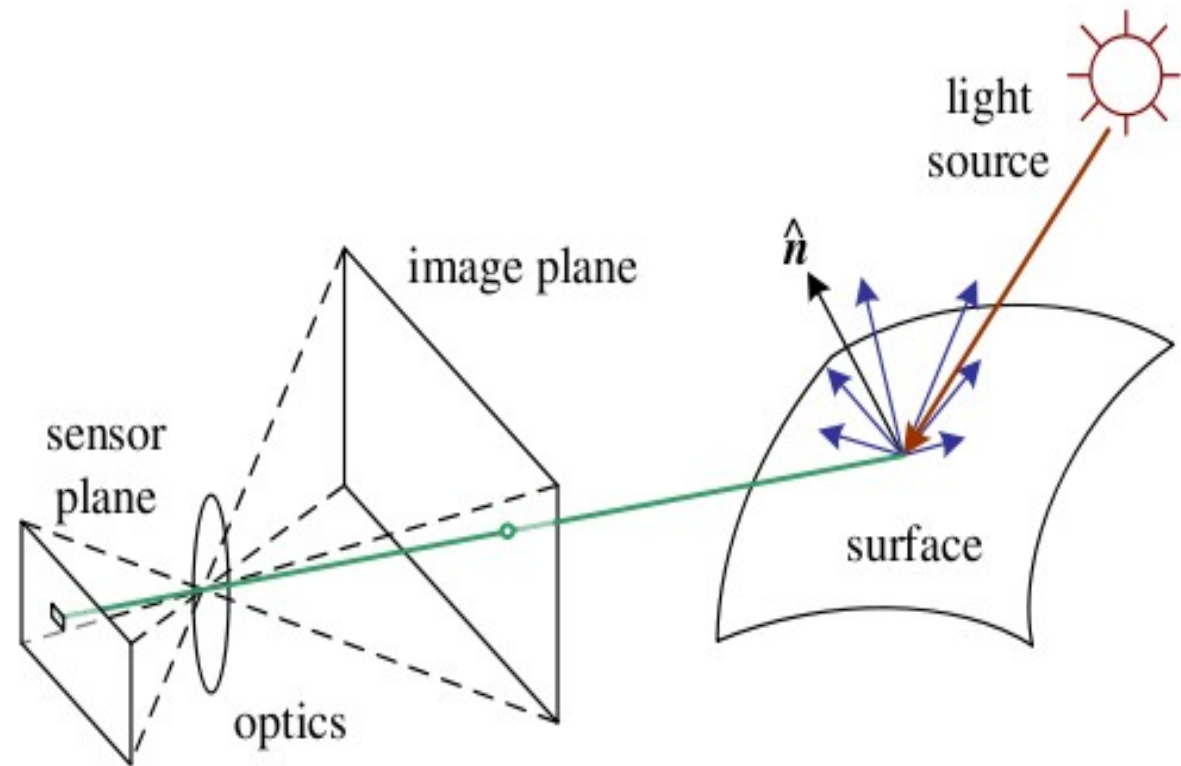
POLITÉCNICA

Índice

1. Introducción
2. Transformaciones puntuales
 - Transformaciones basadas en el histograma
 - Ecualización del histograma
3. Transformaciones basadas en ventanas
 - Filtrado no lineal de imágenes
 - Filtrado lineal de imágenes
4. Análisis de formas
 - Operador de bordes de Canny
 - Extracción de círculos
 - Orientación de objetos elongados
 - Análisis de la imagen de la línea
5. Descripción de regiones
 - Momentos
 - ORB

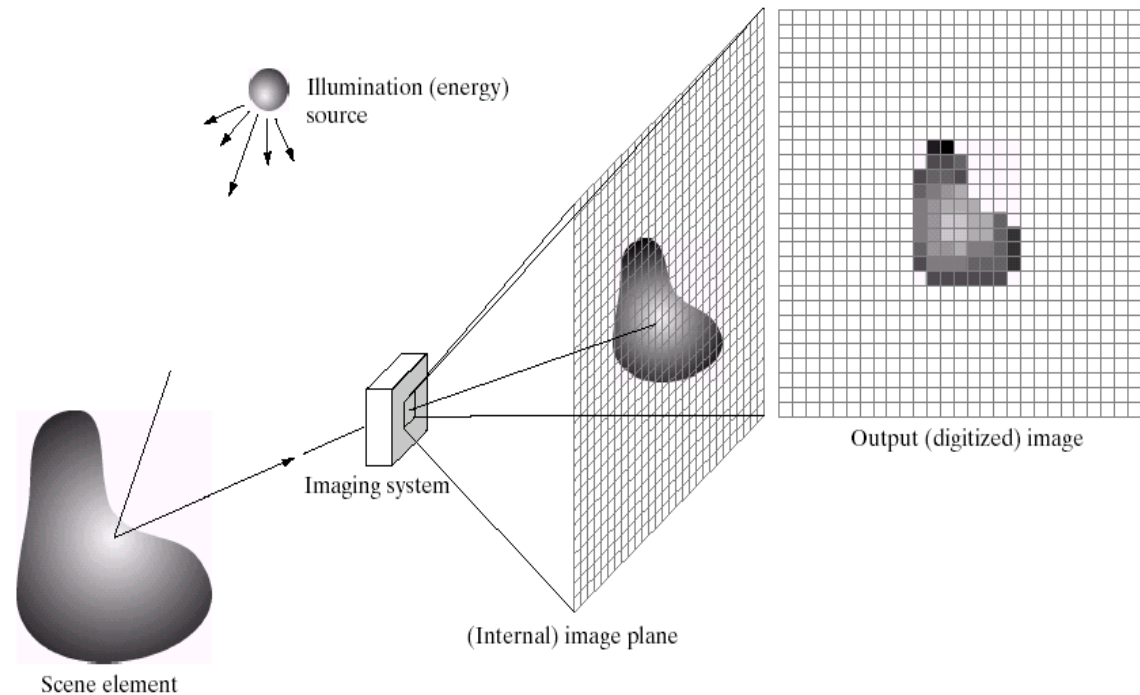
Introducción

- Formación de una imagen digital
- Los niveles de gris de una imagen dependen:
- Condiciones de iluminación
 - Geometría de la escena
 - Propiedades de la superficie
 - Cámara



Introducción

- Formación de una imagen digital

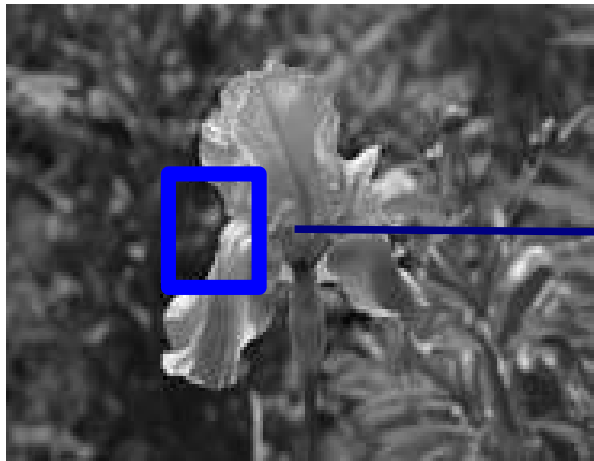


Digital image Processing. Second Edition. R. Gozalez y R. Woods.

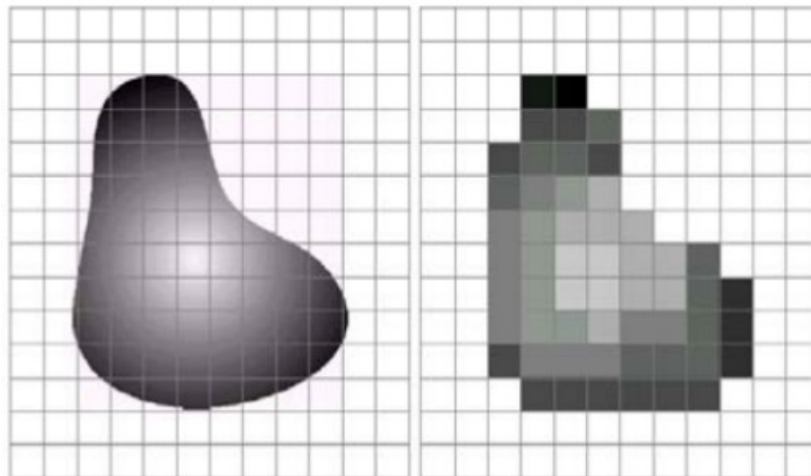
Es una representación matricial obtenida a partir de un **muestreo** en el dominio del espacio y de una **cuantificación** del color/nivel de gris, dando lugar a un conjunto de píxeles que toman valores en un dominio discreto.

Introducción

- Formación de una imagen digital
 1. Muestrea la retina en una cuadrícula regular
 2. Cuantifica la carga de cada píxel

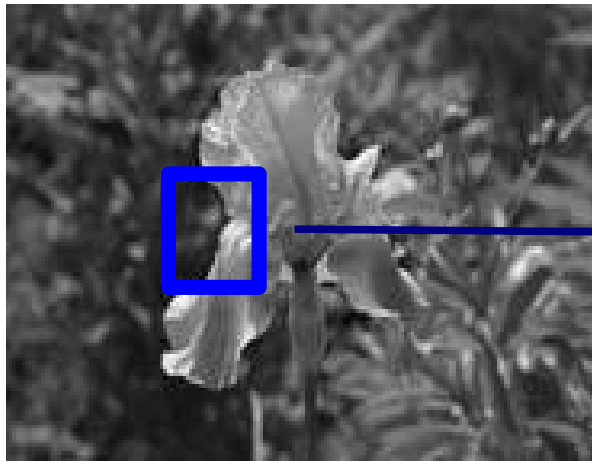


45	60	98	127	132	133	137	133
46	65	98	123	126	128	131	133
47	65	96	115	119	123	135	137
47	63	91	107	113	122	138	134
50	59	80	97	110	123	133	134
49	53	68	83	97	113	128	133
50	50	58	70	84	102	116	126
50	50	52	58	69	86	101	120



Introducción

- Formación de una imagen digital
 1. Muestrea la retina en una cuadrícula regular
 2. Cuantifica la carga de cada píxel



45	60	98	127	132	133	137	133
46	65	98	123	126	128	131	133
47	65	96	115	119	123	135	137
47	63	91	107	113	122	138	134
50	59	80	97	110	123	133	134
49	53	68	83	97	113	128	133
50	50	58	70	84	102	116	126
50	50	52	58	69	86	101	120

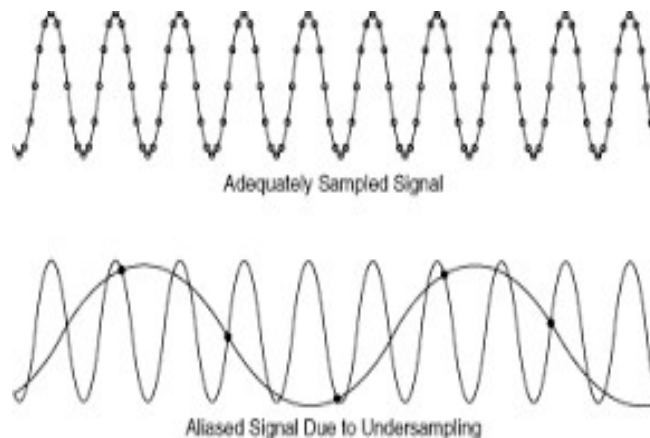
Este proceso de formación de una imagen da lugar a algunos efectos indeseados:

- aliasing
- mal contraste
- ruido

Introducción

Aliasing

- El muestreo espacial de la imagen se realiza con una frecuencia (inversa de la distancia entre los píxeles) finita.
- Cuando la imagen está compuesta por estructuras repetitivas, si la frecuencia de muestreo es menor que el doble de la frecuencia de la imagen (teorema de Nyquist-Shannon), entonces aparecen en la imagen otros patrones conocidos como “aliasing”



Introducción

Veremos técnicas de procesamiento de imágenes que transforman una imagen en otra que:

- tiene una mejor **distribución en los niveles de gris** (transformaciones puntuales)



Introducción

El procesamiento de imágenes digitales transforma una imagen en otra que:

- tiene una mejor **distribución en los niveles de gris** (transformaciones puntuales)
- tiene menos **ruido** (suavizado o filtrado paso bajo)



Introducción

El procesamiento de imágenes digitales transforma una imagen en otra que:

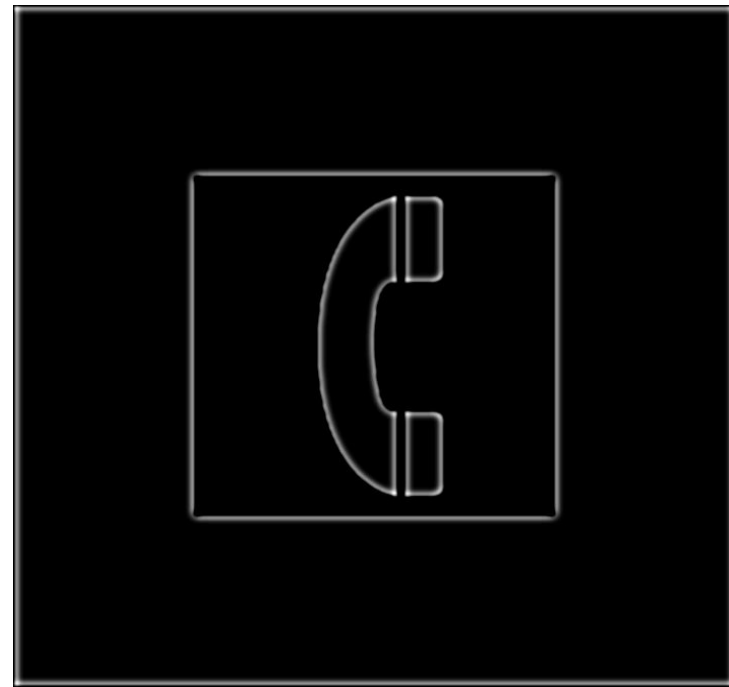
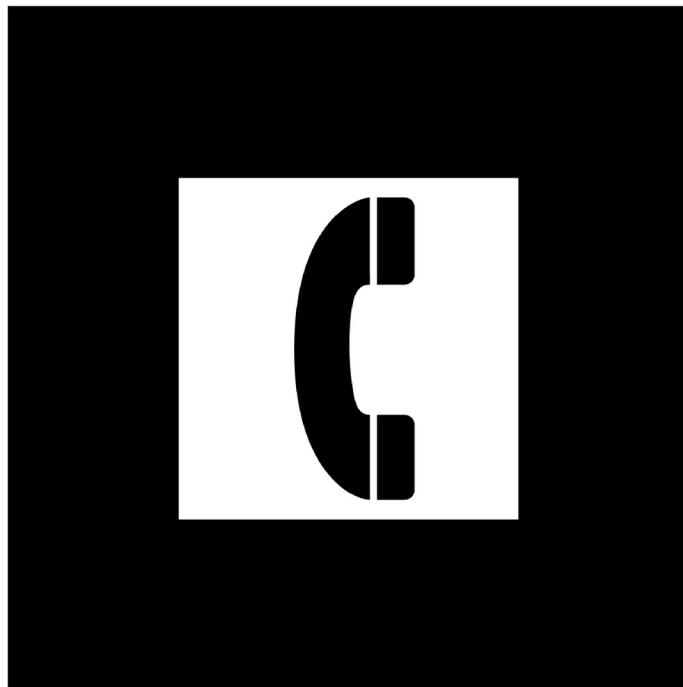
- tiene una mejor **distribución en los niveles de gris** (transformaciones puntuales)
- tiene menos **ruido** (suavizado o filtrado paso bajo)
- algunas estructuras de la imagen aparecen realzadas (filtrado paso alto)



Introducción

El procesamiento de imágenes digitales transforma una imagen en otra que:

- tiene una mejor **distribución en los niveles de gris** (transformaciones puntuales)
- tiene menos **ruido** (suavizado o filtrado paso bajo)
- algunas estructuras de la imagen aparecen realzadas (filtrado paso alto)



Introducción

- Imagen digital

Sea $\mathcal{F}$ el conjunto de las filas $\mathcal{C}$ el de las columnas

$\mathcal{F} = \{0, \dots, f - 1\}$ y $f \times c$ la **resolución espacial**.

$\mathcal{C} = \{0, \dots, c - 1\}$

Una imagen digital es una función

$$I : \mathcal{F} \times \mathcal{C} \rightarrow \mathcal{D}$$

siendo

$$\mathcal{D} = \{0, \dots, d - 1\}$$

$$d = 256$$

la **resolución digital** de la imagen,
habitualmente

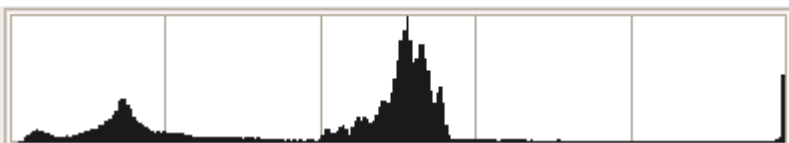
	0							$c - 1$
0	45	60	98	127	132	133	137	133
	46	65	98	123	126	128	131	133
	47	65	96	115	119	123	135	137
	47	63	91	107	113	122	138	134
	50	59	80	97	110	123	133	134
	49	53	68	83	97	113	128	133
	50	50	58	70	84	102	116	126
1	50	50	52	58	69	86	101	120

Introducción

- Histograma de una imagen digital

Sea $I(x, y)$ una imagen digital, su histograma

$$h(m) = \text{card}\{(i, j) | I(i, j) = m\}, \forall m = 0, \dots, d - 1$$



Introducción

- Histograma de una imagen digital

Sea $I(x, y)$ una imagen digital, su histograma

$$h(m) = \text{card}\{(i, j) | \mathcal{I}(i, j) = m\}, \forall m = 0, \dots, L-1$$

Propiedades:

1. $\sum_{m=0}^{L-1} h(m)$ es el número de píxeles de la imagen
2. El histograma de la unión de un conjunto de regiones disjuntas $R_1 \cup R_2, \cup \dots \cup R_r$ viene dado por

$$h(m) = \sum_i h_{R_i}$$

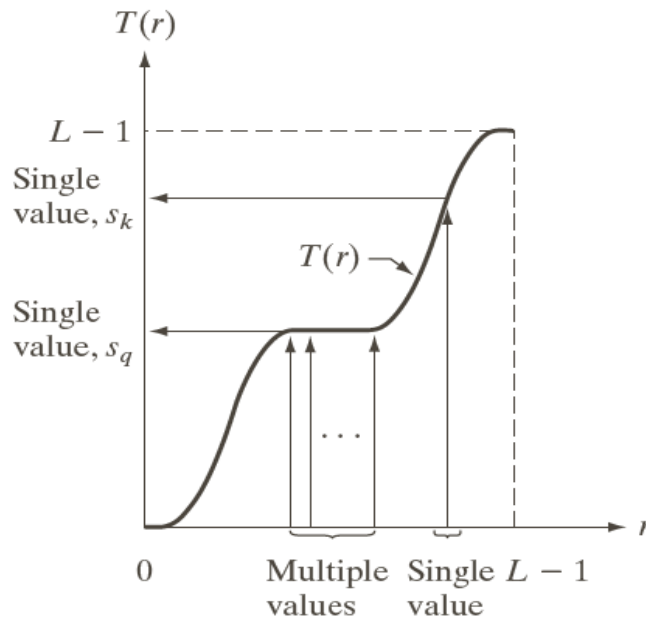
Transformaciones puntuales

Una transformación puntual $\mathcal{T}$ es una función $\mathcal{T} : \mathcal{D} \rightarrow \mathcal{D}$.

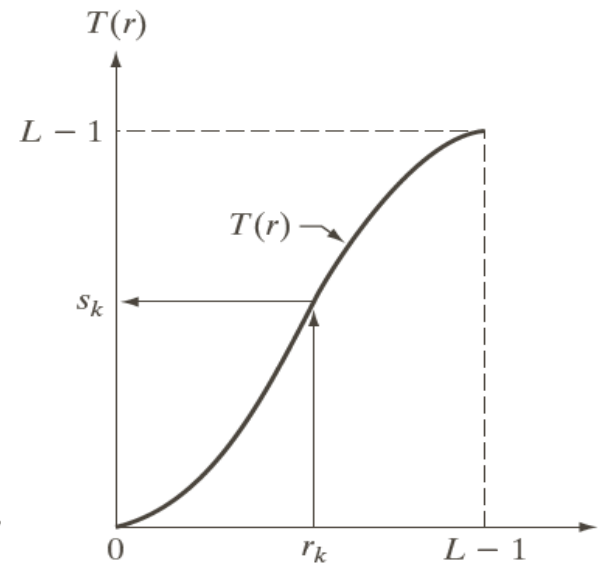
Toma el nivel de gris de un pixel y lo transforma en otro nivel de gris

$$\mathcal{T}[\mathcal{I}(x, y)] = \mathcal{S}(x, y) \in \mathcal{D}$$

Vamos a asumir que $\mathcal{T}$ es una función monótona no decreciente.

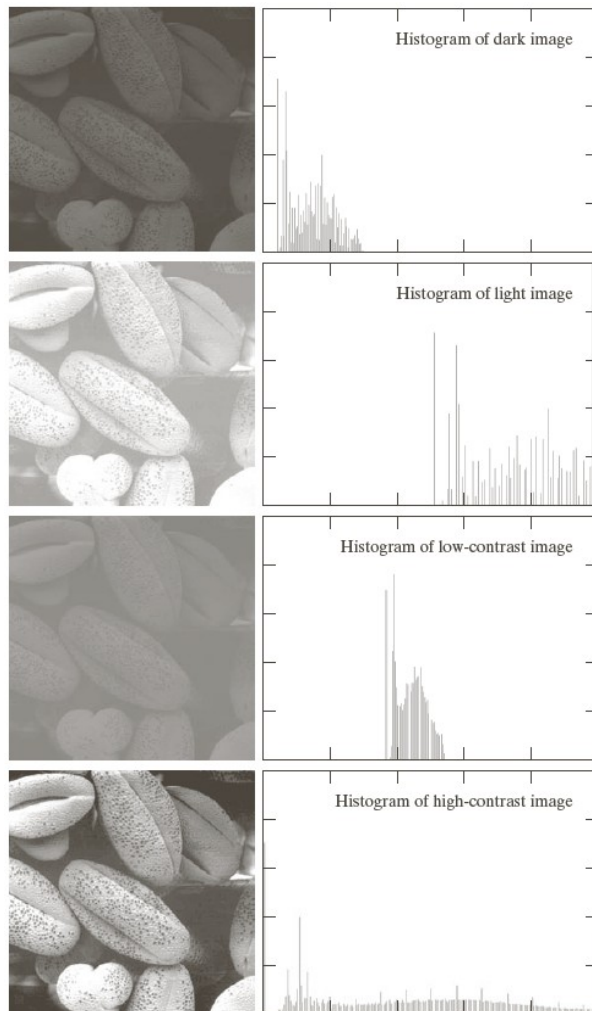


Monótona no
decreciente



Monótona creciente

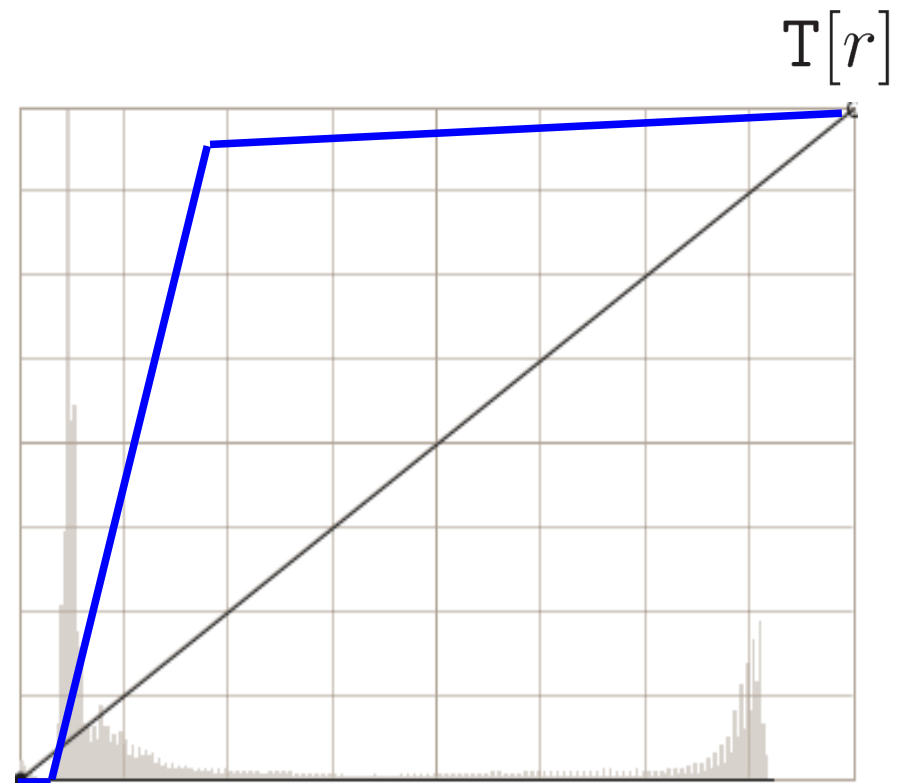
Transformaciones puntuales



Habitualmente deseamos encontrar una transformación que aumente el rango dinámico, y por tanto el contraste, de la imagen.

Transformaciones puntuales

Transformación de una imagen tomada con una iluminación deficiente



Transformaciones puntuales

Transformación de una imagen tomada con una iluminación deficiente

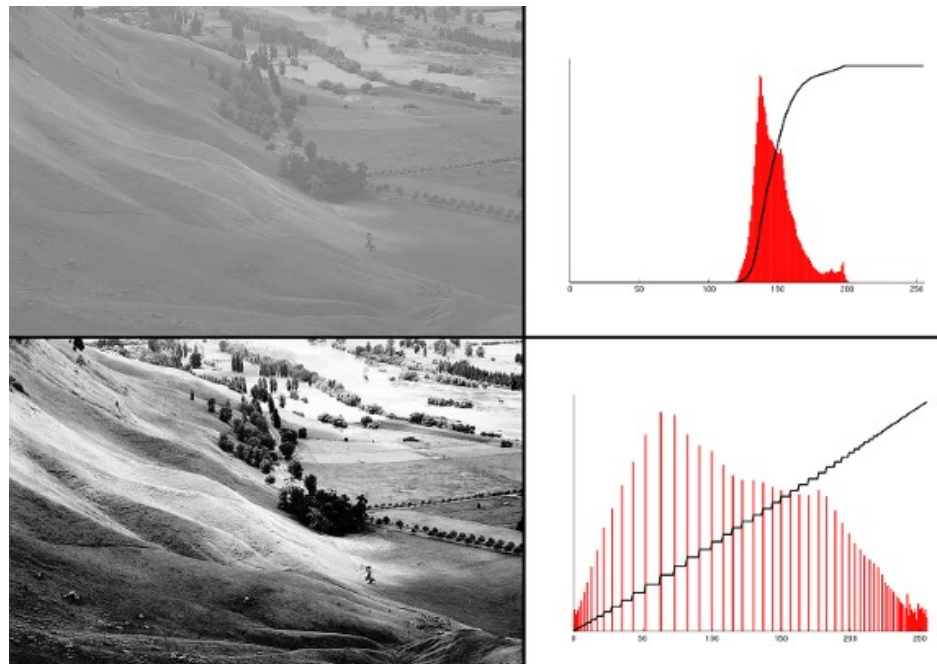


Transformaciones puntuales

Equalización del histograma

Procedimiento automático de cálculo de $T[r]$ de tal forma que en la imagen transformada:

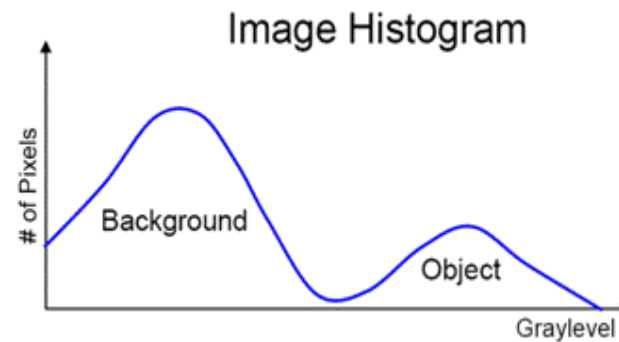
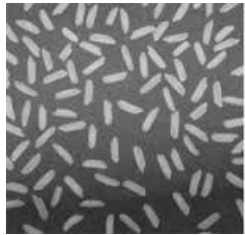
- El rango dinámico sea el máximo
- Los píxeles se distribuyan uniformemente



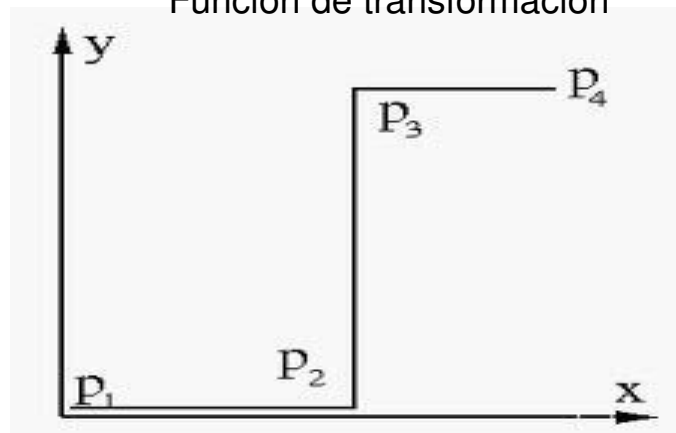
Transformaciones puntuales

Segmentación

En algunas imágenes sencillas se puede utilizar una transformación puntual para segmentar.



Función de transformación



Transformaciones puntuales

Implementación OpenCV

```
import cv2

img = cv2.imread(...)

imgDest = cv2.convertScaleAbs(img, alpha = 1, beta = 0)
```

Parameters:

img – input array, 8 bits element.

alpha – optional scale factor.

beta – optional delta added to the scaled values.

On each element of the input array, the function `convertScaleAbs` performs three operations sequentially: scaling, taking an absolute value, conversion to an unsigned 8-bit type.

In case of multi-channel arrays, the function processes each channel independently.

Transformaciones puntuales

Implementación OpenCV

```
import cv2

img = cv2.imread(...)

imgDest = cv2.LUT(img, lut)
```

Parameters:

img – input array of 8-bit elements.

lut – look-up table of 256 elements;

in case of multi-channel input array, the table should either have a single channel or the same number of channels as in the input array.

Fills the output array with values from the look-up table. The function processes each element of `img` as follows:

where `d=128` si `img` es `int8` y `d=0` si `uint8`

$$\forall i, j \quad imgDest[i, j] = lut[img[i, j] + d]$$

Transformaciones puntuales

Implementación OpenCV

```
import cv2

img = cv2.imread(...)

imgDest = cv2.equalizeHist(img)
```

Parameters:

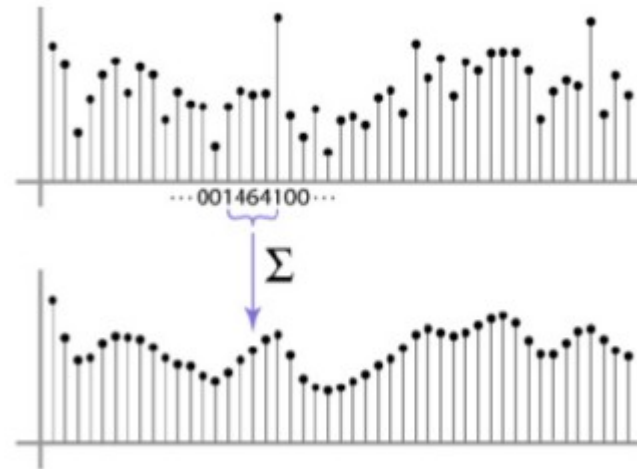
img – input array of 8-bit elements.

Equalizes the histogram of a grayscale image.

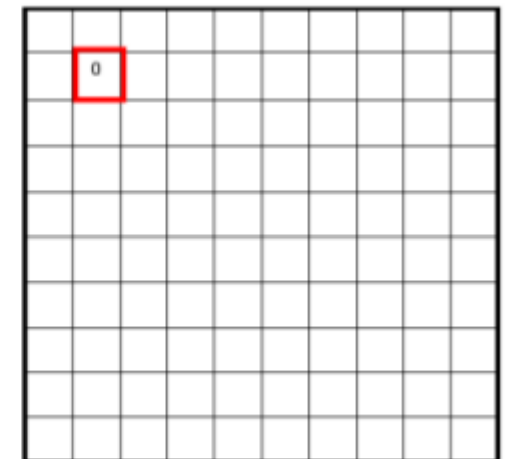
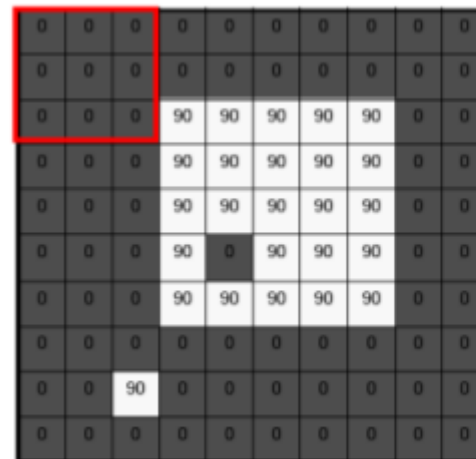
Transformaciones basadas en ventanas

El valor en el que se transforma un píxel depende de su nivel de gris y del de sus vecinos:

- Ventana de vecinos 1-D



- Ventana de vecinos 2-D



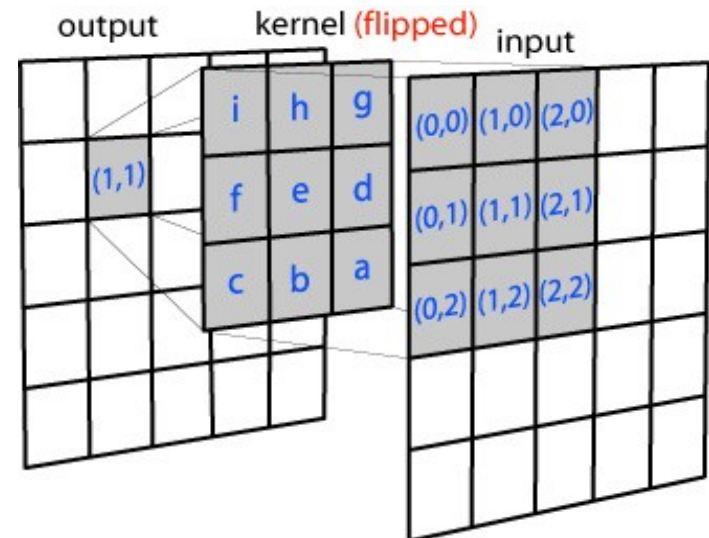
Transformaciones basadas en ventanas

El valor en el que se transforma un píxel depende de su nivel de gris y del de sus vecinos:

- Ventana de vecinos 1-D
- Ventana de vecinos 2-D

Existen dos grupos de transformaciones:

- Transformaciones lineales
 - Los píxeles se transforman mediante una combinación lineal de los valores de los píxeles vecinos.



Transformaciones basadas en ventanas

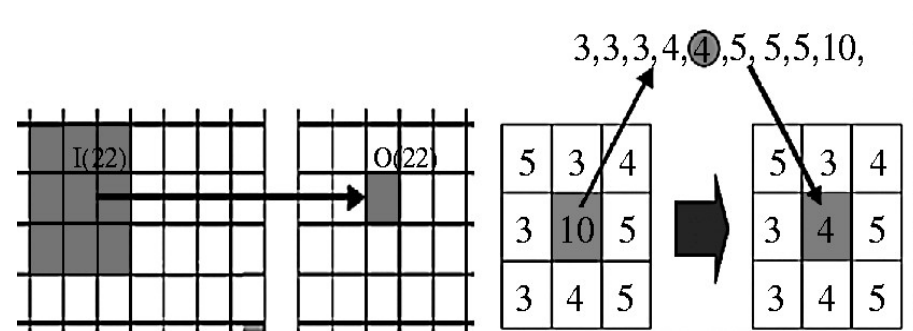
El valor en el que se transforma un píxel depende de su nivel de gris y del de sus vecinos:

- Ventana de vecinos 1-D
- Ventana de vecinos 2-D

Existen dos grupos de transformaciones:

- Transformaciones lineales
- Transformaciones no lineales

La transformación no puede expresarse como una combinación lineal

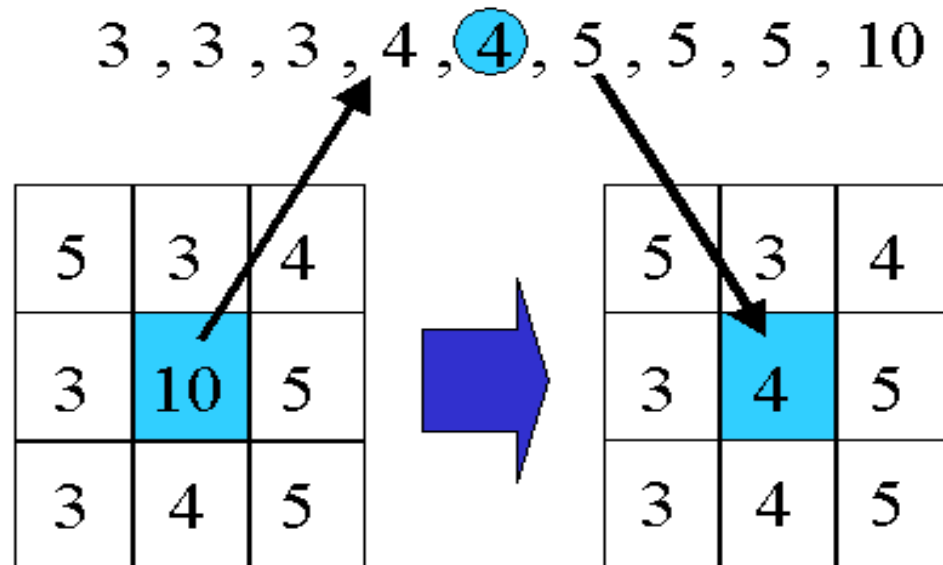


Filtrado no lineal

Filtrado de la mediana

El valor transformado es la mediana de los vecinos.

Filtra bien los valores atípicos.

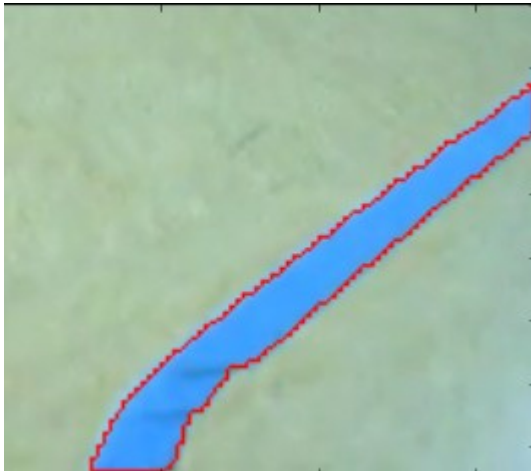


Filtrado no lineal

Filtrado de la mediana

El valor transformado es la mediana de los vecinos.

Filtra bien los valores atípicos.



added noise



median



Filtrado no lineal

Filtrado de la mediana

El valor transformado es la mediana de los vecinos.
Filtra bien los valores atípicos.

- ¿De qué depende la fuerza del filtrado?

Del tamaño de la ventana.

- ¿Qué filtra?

Segmentos cuyo tamaño sea inferior al 50% de la ventana.

- Pero ...

El coste computacional es importante

Filtrado no lineal

Filtrado de la mediana

```
import cv2

img = cv2.imread(...)

imgDest = cv2.medianBlur(img, ksize)
```

Parameters:

img – input 1-, 3-, or 4-channel image (CV_8U, CV_16U, or CV_32F); when ksize is 3 or 5, the image depth should be `uint8`, `uint16`, or `float32`, for larger aperture sizes, it can only be `uint8`.

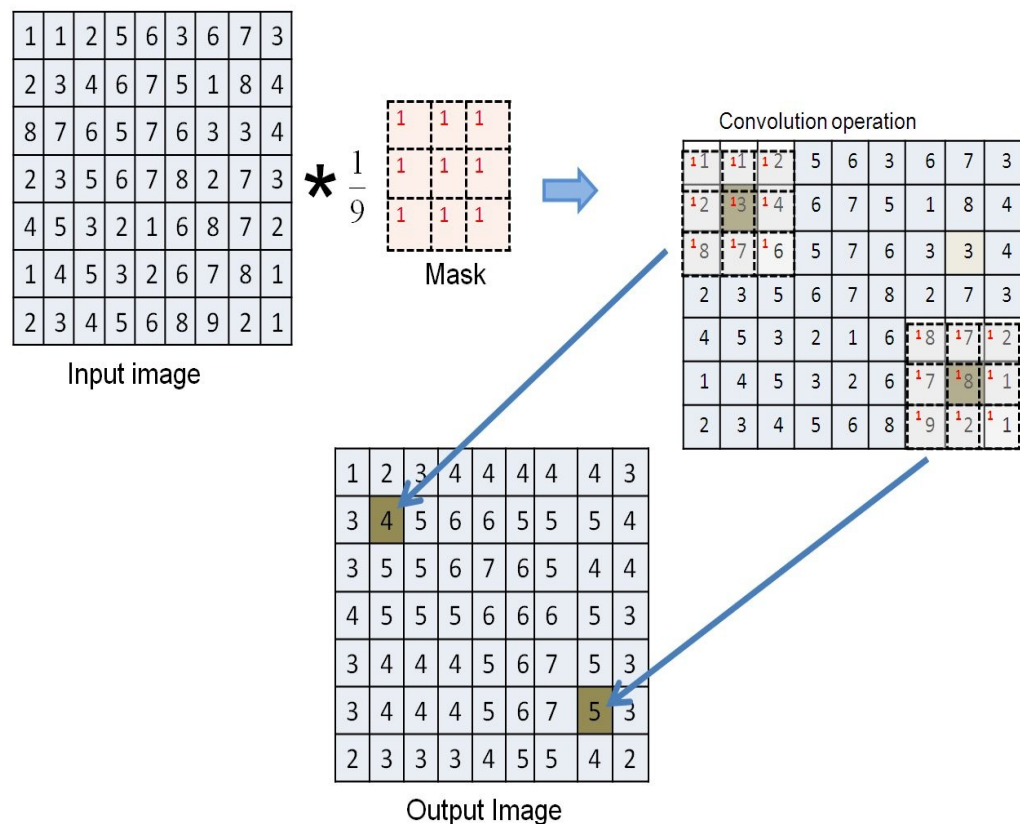
ksize – aperture linear size; it must be odd and greater than 1

The function smoothes an image using the median filter with the `ksize x ksize` aperture. Each channel of a multi-channel image is processed independently.

Filtrado de imágenes digitales

El filtrado lineal se apoya en la operación de **convolución**

Sean e y h dos funciones discretas definidas respectivamente en los dominios $M \times N$ y $m \times n$. La convolución $o = h * e$ es una función de dimensión $M + m - 1 \times N + n - 1$

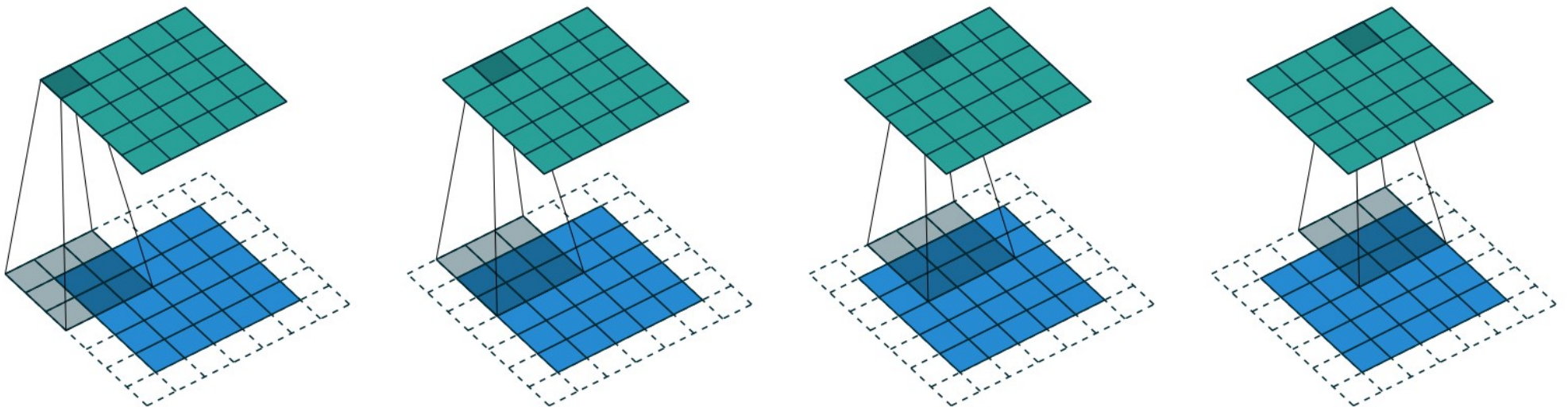


Filtrado de imágenes digitales

El filtrado lineal se apoya en la operación de **convolución**

$$o(r, t) = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} h(i, j) e_e[r + i - (m - 1)/2, t + j - (n - 1)/2]$$

donde o tiene las dimensiones $M \times N$.

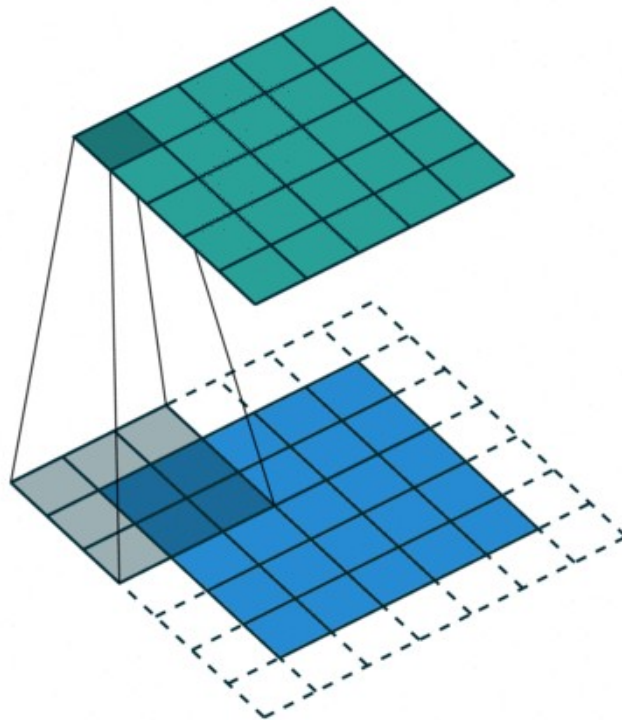


Filtrado de imágenes digitales

El filtrado lineal se apoya en la operación de **convolución**

$$o(r, t) = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} h(i, j) e_e[r + i - (m - 1)/2, t + j - (n - 1)/2]$$

donde o tiene las dimensiones $M \times N$.



Filtrado de imágenes digitales

Filtrado lineal

- Definición

Aquella transformación que puede expresarse como la convolución de la imagen con una máscara o kernel.

Vamos a estudiar dos grupos de filtros:

- **Suavizado (filtrado paso bajo)**

Atenúan el ruido que contamina la imagen.
Se basan en promediar los niveles de gris.

- **Extracción de bordes (filtrado paso alto)**

Realzan las transiciones de niveles de gris.
Se basan en calcular diferencias entre los niveles de gris.

Filtrado de imágenes digitales

Filtrado lineal

Suavizado de imágenes digitales

- Filtro de promediado “tipo caja”

El valor en el que se transforma un píxel es el promedio de sus vecinos.

Por ejemplo:

$$h_c^{3 \times 3} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Filtrado de imágenes digitales

Filtrado lineal

Suavizado de imágenes digitales

- Filtro de promediado “tipo caja”

El valor en el que se transforma un píxel es el promedio de sus vecinos.

Aunque suaviza una imagen, su comportamiento no es perfecto, pues los píxeles cercanos y lejanos influyen de igual manera en el resultado.



Filtrado de imágenes digitales

Filtrado lineal

Suavizado de imágenes digitales

- Filtro de promediado “tipo caja”

```
imgDest = cv2.boxFilter(img, ddepth, ksize [, anchor[, normalize[, borderType]]])
```

Parameters:

img – input image.

ddepth – the output image depth CV_8U, CV_16U, CV_32F, ...(-1 to use `img.depth()`).

ksize – blurring kernel size.

anchor – anchor point; default value `Point(-1,-1)`, the anchor is at the kernel center.

normalize – flag, specifying whether the kernel is normalized by its area or not.

borderType – border mode used to extrapolate pixels outside of the image.

Ejemplo:

```
cv2.boxFilter(img, -1, (7,7), anchor = (-1,-1), normalize = True,  
              borderType = cv2.BORDER_REPLICATE)
```

Filtrado de imágenes digitales

Filtrado lineal

Suavizado de imágenes digitales

- Filtro de promediado “tipo caja” empleando `cv2.filter2D`

```
imgDest = cv2.filter2D(img, ddepth, kernel[, dst[, anchor[, delta[, borderType]]]])
```

Parameters:

kernel – convolution kernel, single channel floating point matrix

delta – optional value added

Ejemplo kernel:

```
img_src = cv2.imread('sample.jpg')

#prepare the 5x5 shaped filter
kernel = np.array([[1, 1, 1, 1, 1],
                   [1, 1, 1, 1, 1],
                   [1, 1, 1, 1, 1],
                   [1, 1, 1, 1, 1],
                   [1, 1, 1, 1, 1]])

kernel = kernel/sum(kernel)
#filter the source image
img_rst = cv2.filter2D(img_src,-1,kernel)
```

Filtrado de imágenes digitales

Filtrado lineal

Tratamiento del borde en filtrado con OpenCV

Enumerator	
BORDER_CONSTANT Python: cv.BORDER_CONSTANT	<code>iiiiii abcdefgh iiiiiii</code> with some specified <code>i</code>
BORDER_REPLICATE Python: cv.BORDER_REPLICATE	<code>aaaaaa abcdefgh hhhhhhh</code>
BORDER_REFLECT Python: cv.BORDER_REFLECT	<code>fedcba abcdefgh hghfedcb</code>
BORDER_WRAP Python: cv.BORDER_WRAP	<code>cdefgh abcdefgh abcdefg</code>
BORDER_REFLECT_101 Python: cv.BORDER_REFLECT_101	<code>ghfedcb abcdefgh gfedcba</code>
BORDER_TRANSPARENT Python: cv.BORDER_TRANSPARENT	<code>uvwxyz abcdefgh ijklmno</code>
BORDER_REFLECT101 Python: cv.BORDER_REFLECT101	same as BORDER_REFLECT_101
BORDER_DEFAULT Python: cv.BORDER_DEFAULT	same as BORDER_REFLECT_101
BORDER_ISOLATED Python: cv.BORDER_ISOLATED	do not look outside of ROI

Filtrado de imágenes digitales

Filtrado lineal

Suavizado de imágenes digitales

- Filtro de suavizado gaussiano

Realiza un promediado ponderado por la función gaussiana

$$g(x) = e^{-\frac{1}{2} \left[\frac{x}{\sigma} \right]^2}$$

donde x es la distancia al píxel filtrado y σ es la desviación típica del filtro.

En este caso, el filtro unidimensional sería

$$h_g^n(x) = e^{-\frac{1}{2} \left[\frac{x - \frac{n-1}{2}}{\sigma} \right]^2}$$

Existe una relación entre n y σ : $n \geq 5\sigma$

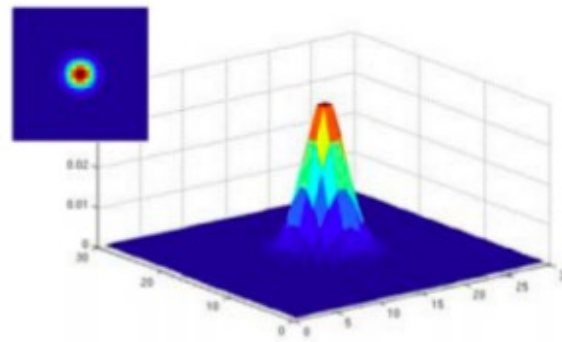
Filtrado de imágenes digitales

Filtrado lineal

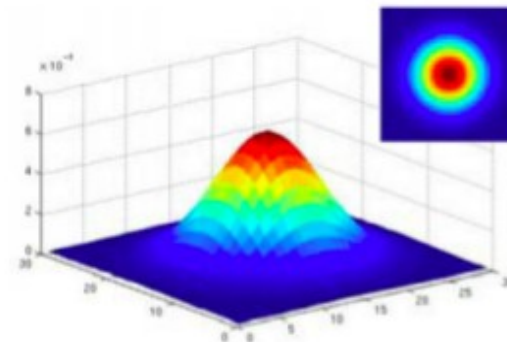
Suavizado de imágenes digitales

- Filtro de suavizado gaussiano

La varianza del filtro determina la fuerza del suavizado



$\sigma = 2$ with
30 x 30
kernel



$\sigma = 5$ with
30 x 30
kernel

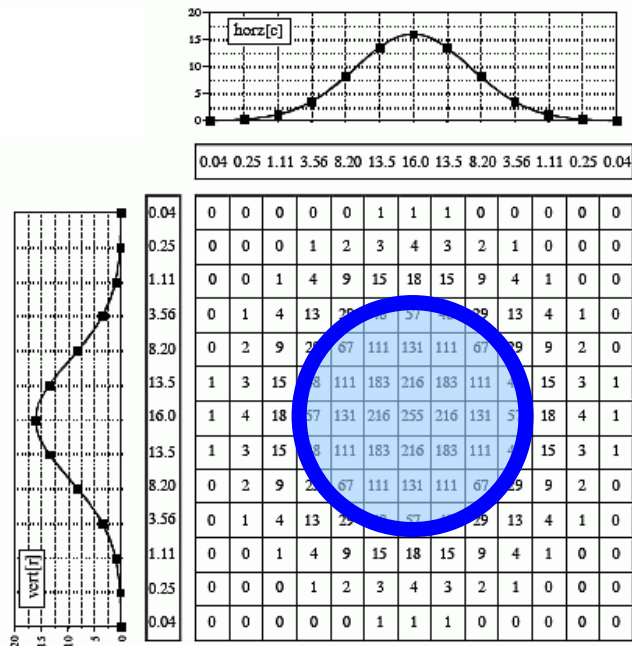
Filtrado de imágenes digitales

Filtrado lineal

Suavizado de imágenes digitales

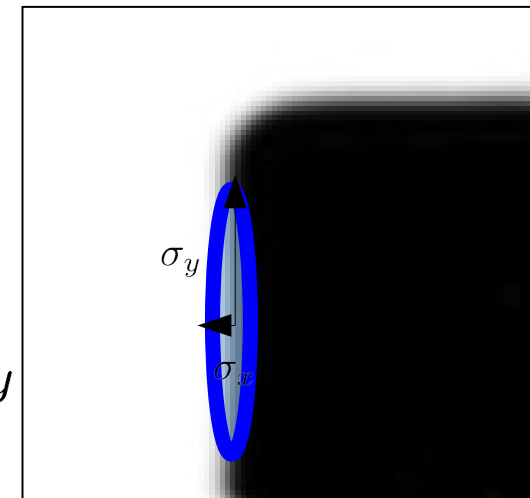
- Filtro de suavizado gaussiano

La forma del filtro puede no ser simétrica



Circular o **isotrópica** (filtra con igual fuerza en todas las direcciones).

$$n = 5\sigma_x$$
$$m = 5\sigma_y$$



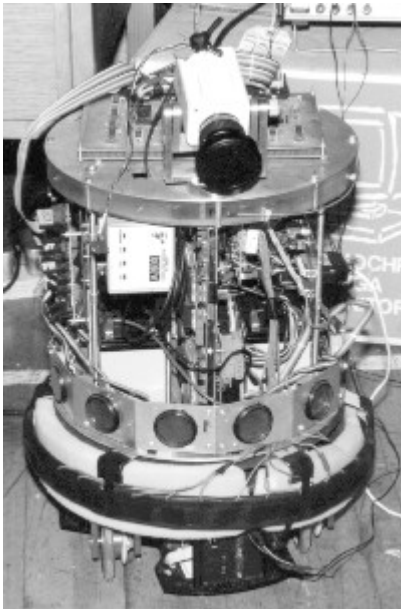
Anisotrópica (la fuerza del filtrado depende de la dirección).

Filtrado de imágenes digitales

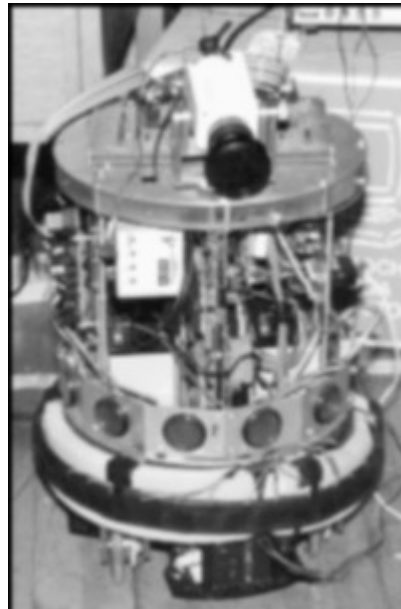
Filtrado lineal

Suavizado de imágenes digitales

- Filtro de suavizado gaussiano



$$\sigma = 0$$



$$\sigma = 1$$



$$\sigma = 2$$



$$\sigma = 4$$

Filtrado de imágenes digitales

Filtrado lineal

Suavizado de imágenes digitales

● Filtro de suavizado gaussiano

```
imgDest = cv2.GaussianBlur(img, ksize, sigmaX[, sigmaY[, borderType]])  
imgDest = cv2.filter2D( ... )
```

Parameters:

- img** – input image; the image can have any number of channels, which are processed independently,
- ksize** – Gaussian kernel size. ksize.width and ksize.height can differ but they both must be positive and odd. Or, they can be zero's and then they are computed from sigma
- sigmaX** – Gaussian kernel standard deviation in X direction.
- sigmaY** – Gaussian kernel standard deviation in Y direction; if sigmaY is zero, it is set to be equal to sigmaX, if both sigmas are zeros, they are computed from ksize.width and ksize.height , respectively. It is recommended to specify all of ksize, sigmaX, and sigmaY.
- borderType** – pixel extrapolation method

Ejemplo:

```
linImgFilGM=cv2.GaussianBlur(linImg, (7,7),0)
```


Filtrado de imágenes digitales

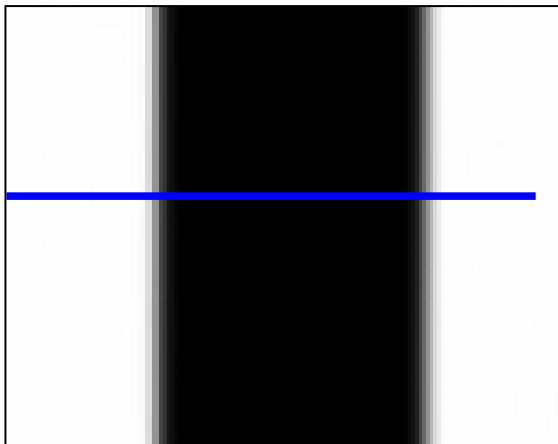
Filtrado lineal

Extracción de bordes

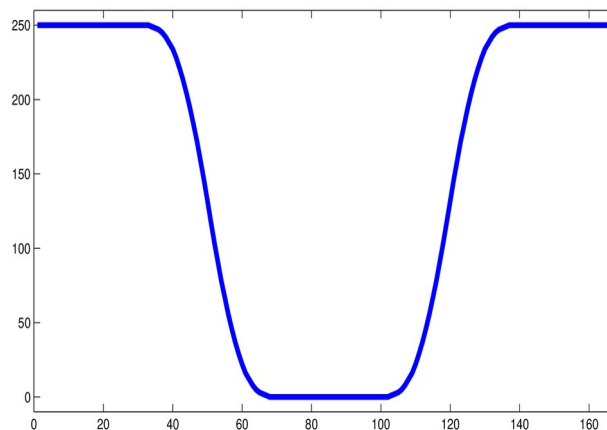
- **Filtro basado en el gradiente**

Los bordes de una imagen están en los máximos del gradiente.

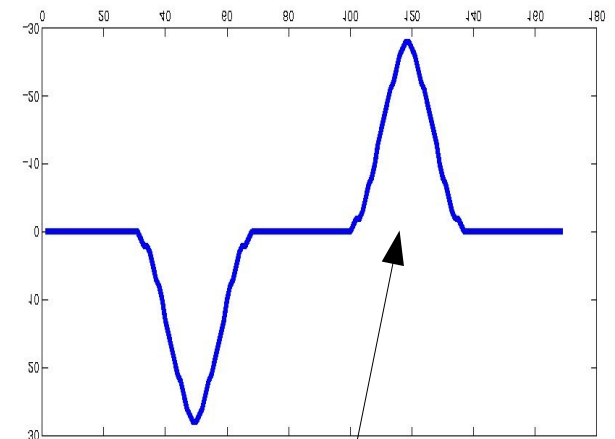
Imagen



Perfil de grises



Derivada



Los bordes coinciden con los extremos de la derivada

Filtrado de imágenes digitales

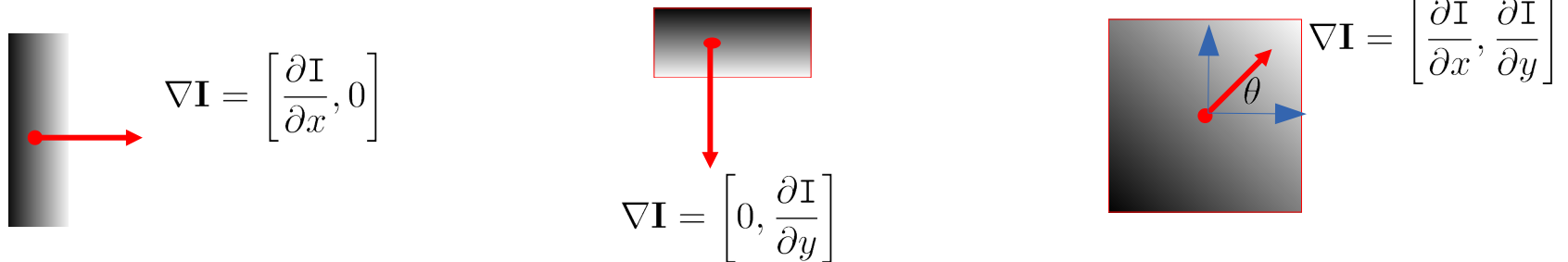
Filtrado lineal

Extracción de bordes

- **Filtro basado en el gradiente**

Los bordes de una imagen están en los máximos del gradiente.

El gradiente de la imagen $I(x, y)$ es un vector que apunta en la dirección de máximo crecimiento de la intensidad



$$|\nabla I| = \sqrt{I_x^2 + I_y^2}$$

$$\theta = \arctan \left(\frac{I_y}{I_x} \right)$$

Filtrado de imágenes digitales

Filtrado lineal

Extracción de bordes

- Filtro basado en el gradiente

Se puede aproximar numéricamente

$$\frac{\partial I(\mathbf{u})}{\partial x} = \lim_{h \rightarrow 0} \frac{I(x+h, y) - I(x-h, y)}{2h} \approx \frac{I(x+1, y) - I(x-1, y)}{2}$$

y expresarse como una convolución de la imagen

$$\frac{\partial I}{\partial x} = \mathbf{h}_{dx} \star I = \frac{1}{2} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * I \quad \frac{\partial I}{\partial y} = \mathbf{h}_{dy} \star I = \frac{1}{2} \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} * I$$

Filtrado de imágenes digitales

Filtrado lineal

Extracción de bordes

- Filtro basado en el gradiente

Los bordes de una imagen están en los máximos del gradiente.

Podemos calcular la derivada de una imagen en la dirección horizontal y/o en la vertical convolucionando la imagen con un kernel de **Sobel**.

El kernel de Sobel combina el filtrado gaussiano y la derivada, de forma que el resultado sea robusto frente al ruido

$$I_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} * I$$

Kernel sobel
horizontal

$$I_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * I$$

Kernel sobel
vertical

Filtrado de imágenes digitales

Filtrado lineal

Extracción de bordes

- Filtro basado en el gradiente, de Sobel

```
dst = cv2.Sobel(img, ddepth, dx, dy[, dst[, ksize[, scale[, delta[, borderType]]]])
```

Parameters:

img – input image

ddepth – depth of output image

ksize – Gaussian kernel size. ksize.width and ksize.height can differ but they both must be positive and odd. Or, they can be zero's and then they are computed from sigma

dx – order of the derivative x.

dy – order of the derivative y.

scale – optional scale value

delta - optional delta value that is added to the result

borderType – pixel extrapolation method

Filtrado de imágenes digitales

Filtrado lineal

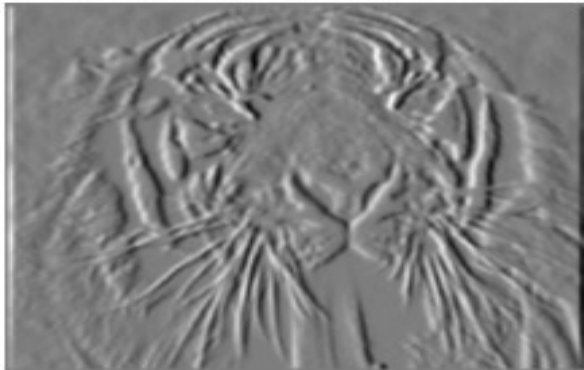
Extracción de bordes

- Filtro basado en el gradiente

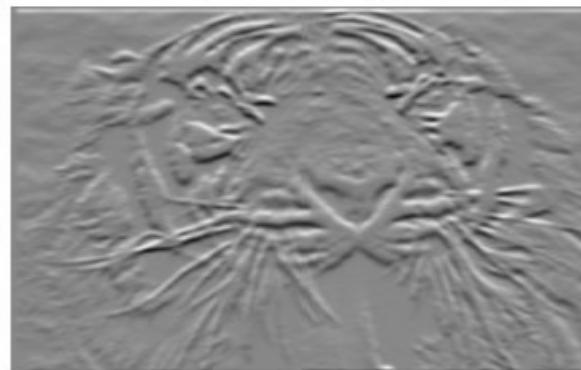
I



$|\nabla I|$



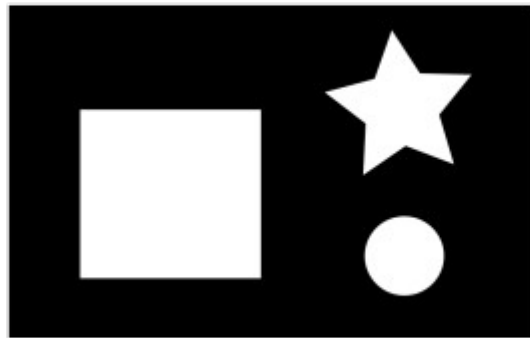
$$I_x = h_{dx} * I$$



$$I_y = h_{dy} * I$$

Análisis de imágenes digitales

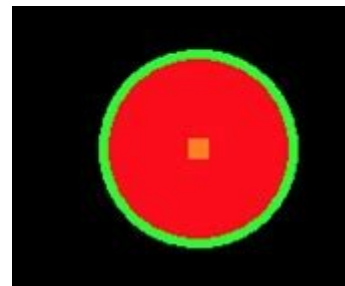
Algoritmos que permiten extraer de una imagen información útil para interpretarla.



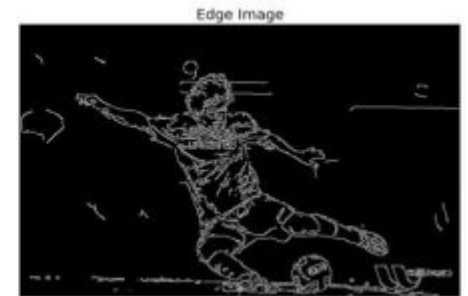
Buscar contornos



Cierre convexo



Detección
círculos



Extracción de
bordes



Agujeros en
el cierre

Análisis de imágenes digitales en OpenCV

- **Operador de bordes de Canny**

Método de detección de bordes delgados (1 píxel de grosor) y supresión de ruido.

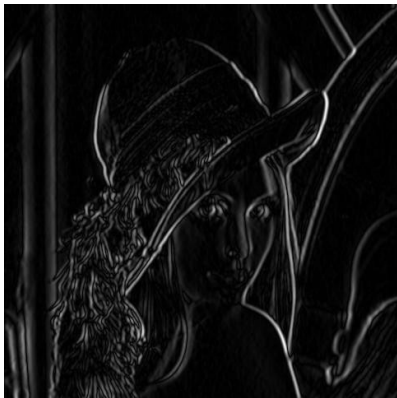


Análisis de imágenes digitales en OpenCV

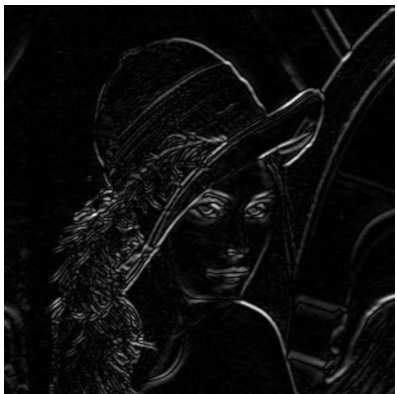
● Operador de bordes de Canny

Método de detección de bordes delgados.

1. Suavizado gaussiano para eliminar ruido
2. Filtrado de Sobel para resaltar los bordes

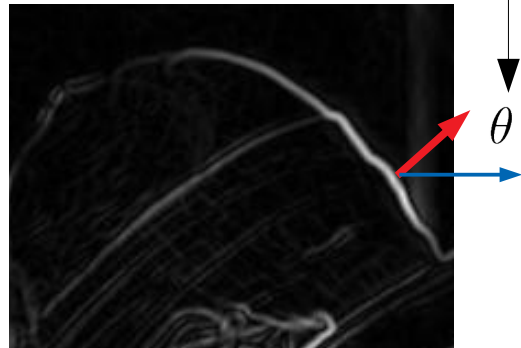


I_x



I_y

$$\theta = \arctan\left(\frac{I_y}{I_x}\right)$$



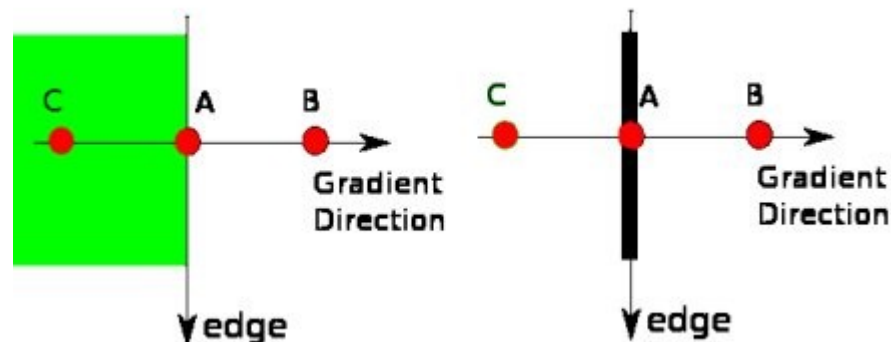
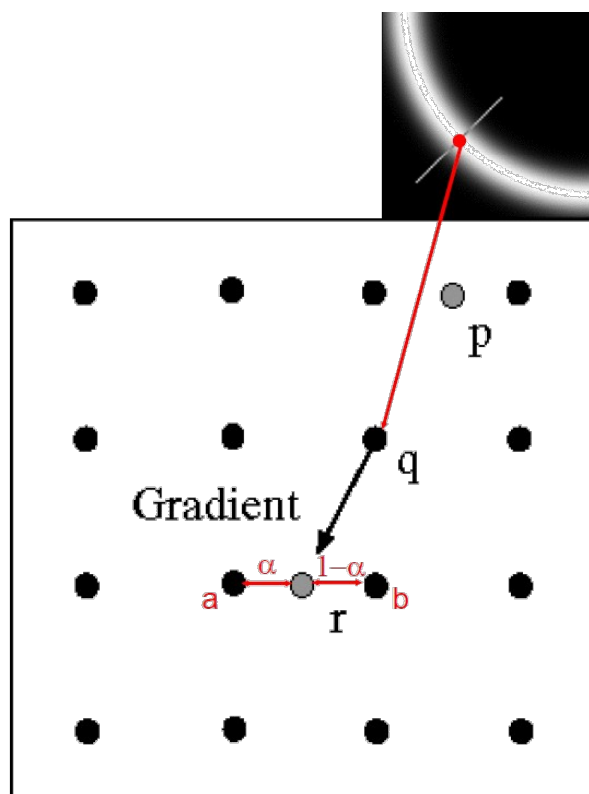
$$|\nabla I| = \sqrt{I_x^2 + I_y^2}$$

Análisis de imágenes digitales en OpenCV

● Operador de bordes de Canny

Método de detección de bordes delgados.

1. Suavizado gaussiano para eliminar ruido
2. Filtrado de Sobel para resaltar los bordes
3. Eliminación de valores no máximos locales



Análisis de imágenes digitales en OpenCV

● Operador de bordes de Canny

Método de detección de bordes delgados.

1. Suavizado gaussiano para eliminar ruido
2. Filtrado de Sobel para resaltar los bordes
3. Eliminación de valores no máximos locales



Análisis de imágenes digitales en OpenCV

● Operador de bordes de Canny

Método de detección de bordes delgados.

1. Suavizado gaussiano para eliminar ruido
2. Filtrado de Sobel para resaltar los bordes
3. Eliminación de valores no máximos locales
4. Doble umbralización



Análisis de imágenes digitales en OpenCV

● Operador de bordes de Canny

Método de detección de bordes delgados.

1. Suavizado gaussiano para eliminar ruido
2. Filtrado de Sobel para resaltar los bordes
3. Eliminación de valores no máximos locales
4. Doble umbralización
5. Seguimiento de bordes



Análisis de imágenes digitales en OpenCV

● Operador de bordes de Canny

Implementación en OpenCV

```
can_edg = cv2.Canny(img, threshold1, threshold2[, edges[, apertureSize[, L2gradient]]])
```

Parameters:

image – Source, img

thres1 – first threshold for the hysteresis procedure.

thres2 – second threshold for the hysteresis procedure

edges – Output edge map

apertureSize – Sobel window size

L2gradient - a flag, indicating whether the correct L2 norm or an approximation should be used

Análisis de imágenes digitales en OpenCV

Búsqueda de formas aproximadamente circulares en una imagen

```
blobs = cv2.SimpleBlobDetector_create(    [, parameters]    )
```

Detecta manchas (“blobs”) y las filtra según parámetros de forma.

```
# Read image  
im = cv2.imread("blob_detection.png", cv2.IMREAD_GRAYSCALE)
```

```
params = cv2.SimpleBlobDetector_Params()
```

```
# Create a detector with the parameters  
detector = cv2.SimpleBlobDetector_create(params)
```

```
# Detect blobs.  
keypoints = detector.detect(im)
```

```
# Setup SimpleBlobDetector parameters.  
params = cv2.SimpleBlobDetector_Params()  
# Change thresholds  
params.minThreshold = 50  
params.maxThreshold = 100  
# Filter by Area.  
params.filterByArea = True  
params.minArea = 100  
# Filter by Circularity  
params.filterByCircularity = True  
params.minCircularity = 1  
# Filter by Convexity  
params.filterByConvexity = False  
params.minConvexity = 0.87  
# Filter by Inertia  
params.filterByInertia = False  
params.minInertiaRatio = 0.01
```



Análisis de imágenes digitales en OpenCV

Búsqueda de formas aproximadamente circulares en una imagen

```
blobs = cv2.SimpleBlobDetector_create(    [, parameters]    )
```

Detecta manchas (“blobs”) y las filtra según parámetros de forma.

```
# Read image  
im = cv2.imread("blob_detection.png", cv2.IMREAD_GRAYSCALE)
```

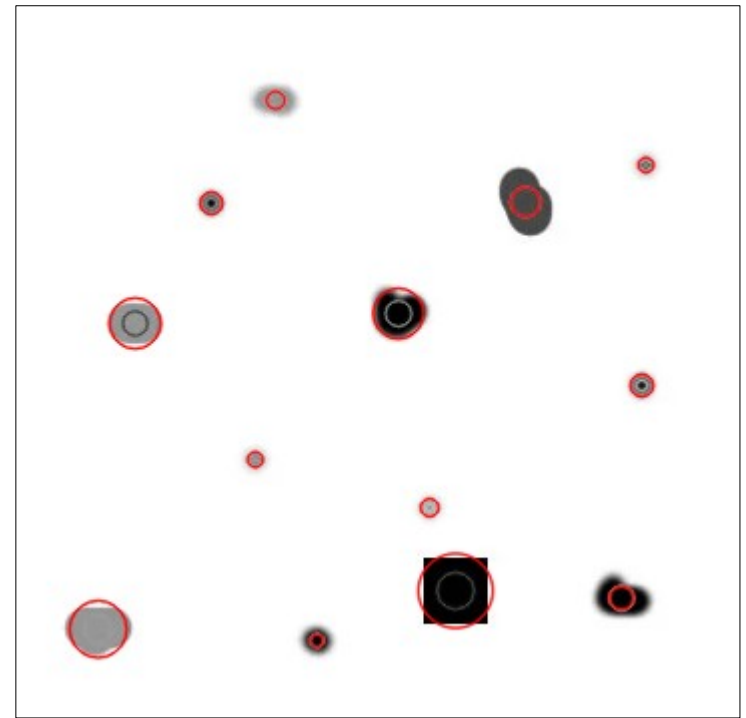
```
params = cv2.SimpleBlobDetector_Params()
```

```
# Create a detector with the parameters  
detector = cv2.SimpleBlobDetector_create(params)
```

```
# Detect blobs.  
keypoints = detector.detect(im)
```

```
# Draw detected blobs as red circles.  
imk = cv2.drawKeypoints(im, keypoints, np.array([]), (0,0,255), \  
                        cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
```

```
# Show blobs  
cv2.imshow("Keypoints", imk)
```



Análisis de imágenes digitales en OpenCV

Búsqueda de formas aproximadamente circulares en una imagen

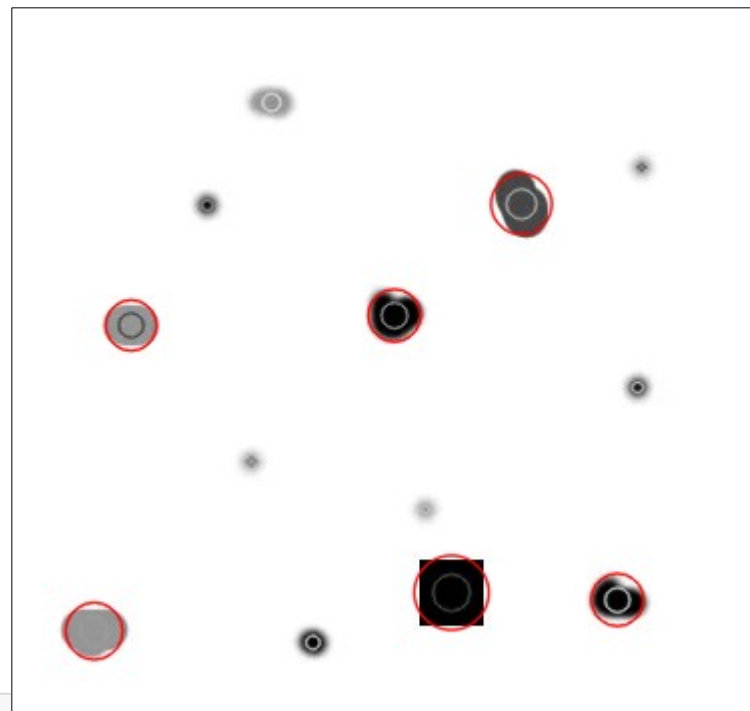
```
blobs = cv2.SimpleBlobDetector_create(    [, parameters]    )
```

Detecta manchas (“blobs”) y las filtra según parámetros de forma.

```
# Setup SimpleBlobDetector parameters.  
params = cv2.SimpleBlobDetector_Params()  
# Change thresholds  
# params.minThreshold = 245  
# params.maxThreshold = 255  
# Filter by Area.  
params.filterByArea = True  
params.minArea = 400  
# Filter by Circularity  
params.filterByCircularity = False  
params.minCircularity = .9  
# Filter by Convexity  
params.filterByConvexity = False  
params.minConvexity = 0.87  
# Filter by Inertia  
params.filterByInertia = False  
params.minInertiaRatio = 0.01
```

```
[(pt.size, pt.pt) for pt in keypoints]
```

```
[(29.97520637512207, (541.2667236328125, 322.6344909667969)),  
(37.161808013916016, (226.5, 301.0)),  
(26.23682975769043, (198.01829528808594, 162.5431365966797)),  
(30.246173858642578, (261.4381103515625, 106.93004608154297)),  
(26.272506713867188, (309.3609313964844, 304.5928649902344)),  
(28.516759872436523, (48.03953552246094, 320.16009521484375)),  
(25.413280487060547, (66.47809600830078, 167.48410034179688))]
```



Análisis de imágenes digitales en OpenCV

Búsqueda de contornos en una imagen

```
conts, hier = cv2.findContours(image, mode, method [, contours [, offset]])
```

Parameters:

image – Source, an 8-bit single-channel image. Non-zero pixels are 1's. The image is treated as binary .

The function modifies the image while extracting the contours (versions <= 2.4).

conts – Detected contours. Each contour is stored as a vector of points.

hier – Optional output vector, containing information about the image topology.

mode – Contour retrieval mode

RETR_EXTERNAL retrieves only the extreme outer contours.

RETR_LIST retrieves all of the contours without establishing any hierarchical relationships.

RETR_CCOMP retrieves all of the contours and organizes them into a two-level hierarchy.

RETR_TREE retrieves all of the contours and reconstructs a full hierarchy of nested contours.

method – Contour approximation method

CHAIN_APPROX_NONE stores absolutely all the contour points.

CHAIN_APPROX_SIMPLE compresses horizontal, vertical, and diagonal segments

CHAIN_APPROX_TC89_L1, CV_CHAIN_APPROX_TC89_KCOS applies chain approximation algorithm

offset – Optional offset by which every contour point is shifted.

Ejemplo:

Construyo una imagen a partir de las etiquetas de segmentación y extraigo los contornos

```
linImg = (labels==2).astype(uint8)[90:,:]*255
```

```
contList, hier = cv2.findContours(linImg, cv2.RETR_LIST, cv2.CHAIN_APPROX_NONE)
```

Análisis de imágenes digitales en OpenCV

Algunas funciones sobre contornos que pueden ser útiles:

```
imgDest = cv2.drawContours(image, contours, contourIdx, color[, thickness[,  
                                lineType[, hierarchy[, maxLevel[, offset]]]]])  
  
approxCurve = cv2.approxPolyDP(curve, epsilon, closed)  
len = cv2.arcLength(curve, closed)  
area = cv2.contourArea(contour)  
  
chull = cv2.convexHull(points[, hull[, clockwise[, returnPoints]]])
```

Ejemplos:

Pinto en rojo los contornos extraídos de una imagen

```
cv2.drawContours(imDraw, contList, -1, (0,0,255))
```

Recorro la lista de contornos y construyo las curvas de los cierres conexos

```
chullList=[cv2.convexHull(cont,returnPoints=False) for cont in contList]
```

Análisis de imágenes digitales en OpenCV

Obtener los “agujeros” en el cierre conexo de un contorno

```
imgDest = cv2.convexityDefects(contour, convexHull)
```

Parameters:

contour – Contour detected with `cv2.findContour`.

convexHull – Convex hull obtained with `cv2.convexHull(..., returnPoints = False)`

Ejemplo:

```
# Busco los agujeros en los contornos calculados anteriormente
convDefs = [cv2.convexityDefects(cont, hull) for (cont,hull) in
              zip(contList,hullList)]

# Selecciono el primer contorno y sus agujeros
cont = contList[0]
cnvDef = convDefs[0]

# Saco la lista de agujeros del contorno
listConvDefs=cnvDef[:,0,:].tolist()

# Devuelvo la lista de agujeros mayores de 4 pixeles, aproximadamente
[[init,end,mid,length] for init,end,mid,length in listConvDefs if length>1000]
```

Análisis de imágenes digitales en OpenCV

Obtener información sobre conjuntos de puntos/contornos.

```
rect = cv2.fitEllipse(points)
```

Parameters:

points – Nx2 Numpy array

Returns: It returns the rotated rectangle in which the ellipse is inscribed.

```
rect = cv2.minAreaRect(points)
```

Parameters:

points – Nx2 Numpy array

Returns: [center (x,y), (width, height), angle_of_rotation]

```
box = cv2.boxPoints(rect)
box = np.int0(box)
cv2.drawContours(img, [box], 0, (0,0,255), 2)
```

Análisis de imágenes digitales en OpenCV

Obtener información sobre conjuntos de puntos/contornos.

```
line = cv2.fitLine(points, distType, param, reps, aeps[, line])
```

Parameters:

points – Nx2 Numpy array

distType - Distance used by the M-estimator (CV_DIST_L2, ... CV_DIST_HUBER)

param - Numerical parameter (C) for some types of distances. If it is 0, an optimal value is chosen.

reps - Sufficient accuracy for the radius (distance between the coordinate origin and the line).

aeps - Sufficient accuracy for the angle. (0.01 would be a good default value for `reps` and `aeps`).

Returns: In case of 2D fitting, it should be a vector of 4 elements (like Vec4f) - (vx, vy, x0, y0), where (vx, vy) is a normalized vector collinear to the line and (x0, y0) is a point on the line.

Descripción de regiones

Objetivo:

Describir imágenes de forma invariante a cambios de orientación, iluminación, escala, etc.

Viewpoint variation



Scale variation



Deformation



Occlusion



Illumination conditions



Background clutter



Intra-class variation



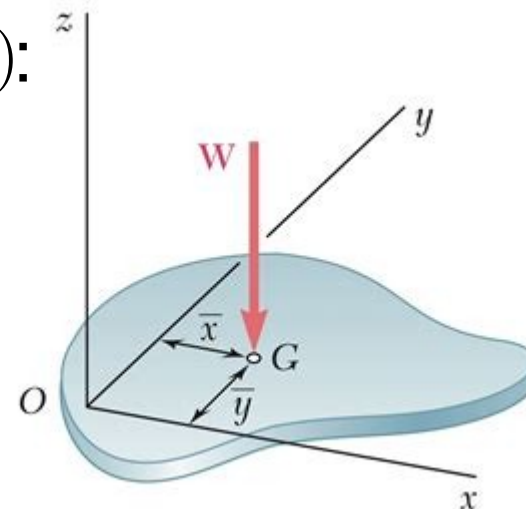
Análisis de imágenes digitales en OpenCV

Los **momentos** describen una imagen $I(x, y)$:

$$M_{ij} = \sum_x \sum_y x^i y^j I(x, y)$$

El centro de masas de una región

$$\bar{x} = \frac{M_{10}}{M_{00}} \quad \bar{y} = \frac{M_{01}}{M_{00}}$$



Los **momentos centrales** son invariantes a traslaciones:

$$\mu_{pq} = \sum_x \sum_y (x - \bar{x})^p (y - \bar{y})^q I(x, y)$$

Los **momentos de Hu** son invariantes a traslaciones, rotaciones y escalado:

inv escala:

$$I_1 = \eta_{20} + \eta_{02}$$

$$I_2 = (\eta_{20} - \eta_{02})^2 + 4\eta_{11}^2$$

$$I_3 = (\eta_{30} - 3\eta_{12})^2 + (3\eta_{21} - \eta_{03})^2$$

$$I_4 = (\eta_{30} + \eta_{12})^2 + (\eta_{21} + \eta_{03})^2$$

$$I_5 = (\eta_{30} - 3\eta_{12})(\eta_{30} + \eta_{12})[(\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2] + (3\eta_{21} - \eta_{03})(\eta_{21} + \eta_{03})[3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2]$$

$$I_6 = (\eta_{20} - \eta_{02})[(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2] + 4\eta_{11}(\eta_{30} + \eta_{12})(\eta_{21} + \eta_{03})$$

$$I_7 = (3\eta_{21} - \eta_{03})(\eta_{30} + \eta_{12})[(\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2] - (\eta_{30} - 3\eta_{12})(\eta_{21} + \eta_{03})[3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2].$$

$$\eta_{ij} = \frac{\mu_{ij}}{\left(1 + \frac{i+j}{2}\right)^{\mu_{00}}}$$

Análisis de imágenes digitales en OpenCV

Implementación de los momentos en OpenCV

```
import cv2  
moments = cv2.moments (array[, binaryImage])  
  
huMoments = cv2.HuMoments (moments)
```

Parameters:

array – input image array, 8 bits.

binaryImage – If it is true, all non-zero image pixels are treated as 1's.

moments – image moments.

Implementación ejemplo:

```
imgLabels = SegmentaImagen(img_rgb, clasifEuclid)  
pixMarca = (imgLabels == 2).astype(np.uint8)*255  
desHu = cv2.HuMoments(cv2.moments(pixMarca, True))  
  
print(desHu)  
[ 3.7827806e-01  9.0754829e-02  4.4973038e-03  2.7160207e-03  
 9.47702067e-06  8.01547695e-04 -5.40014669e-07]
```

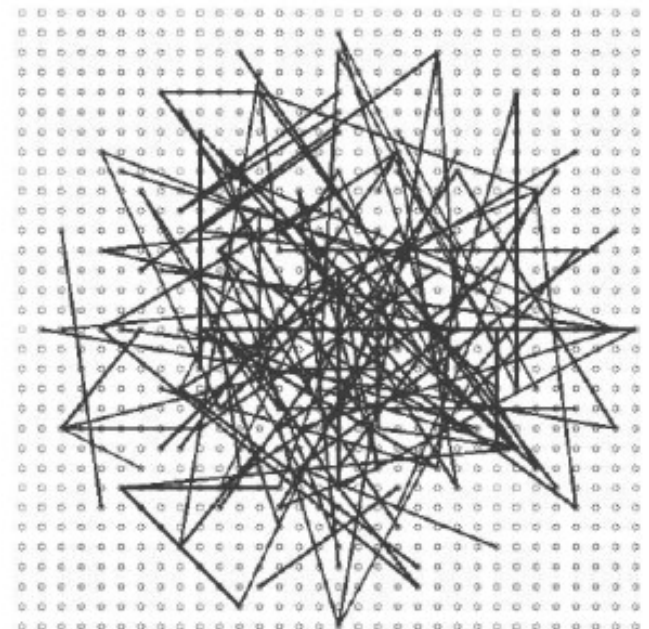
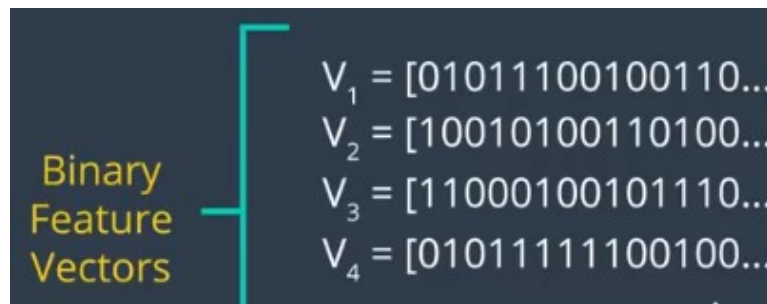
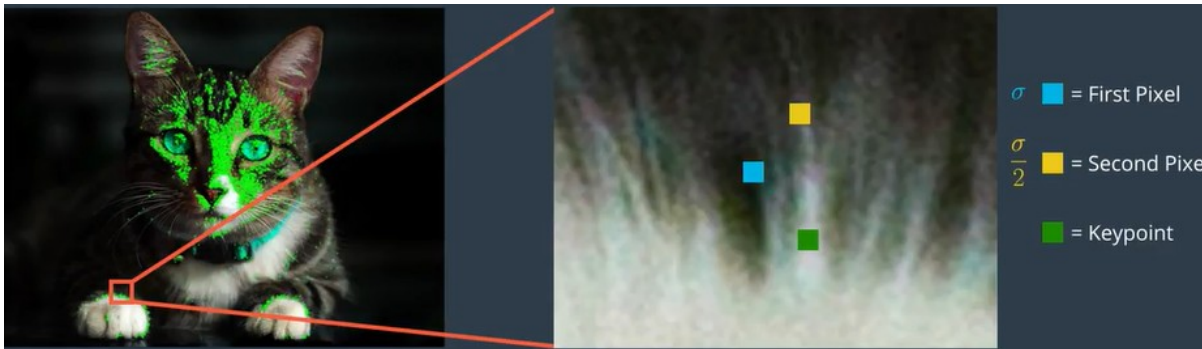
Análisis de imágenes digitales en OpenCV

Descriptor ORB

Descriptor basado en aprendizaje automático

- binario $\mathbf{x} \in \{0, 1\}^m$
- emplea diferencias de niveles de gris, muy rápido de evaluar y emparejar.

Supone un proceso previo de “detección” en el que se establece la escala, orientación y posición de la región.



Análisis de imágenes digitales en OpenCV

Implementación OpenCV ORB

```
import cv2
orb = cv2.ORB_create()
kp = cv2.KeyPoint(cx,cy, diam, ang)
kpList, des = orb.compute(img, kpList)
img2 = cv2.drawKeypoints(img, kpList, None, color=(0,255,0),
flags=0)
```

Parameters:

- cx, cy** – posición del punto a describir
- diam** – diametro de la región circular a describir
- ang** – orientación de la región a describir
- img** – input image array, 8 bits.
- kpList** – List of key points to describe.
- color** – drawing color
- flags** – what to draw.

Análisis de imágenes digitales en OpenCV

Implementación OpenCV ORB

Ejemplo descripción de una región:

```
# Busco los contornos de la marca
```

```
_, contList, hier = cv2.findContours(pixMarca,...)
```

```
# Ajusto elipse para identificar la orientacion
```

```
ellip = cv2.fitEllipse(contList[0])
```

```
cen, ejes, angulo = np.array(ellip[0]), np.array(ellip[1]), ellip[2]
```

```
# Creo un descriptor para describir la marca con ORB.
```

```
# - centro es el centro de la elipse
```

```
# - diametro: es 30% mayor que media semiejes mayores y menores
```

```
# - angulo: es el angulo de la elipse girado apunte hacia arriba
```

```
kp = cv2.KeyPoint(cen[0], cen[1], np.mean(ejes)*1.3, angulo-90)
```

```
# Describe la region
```

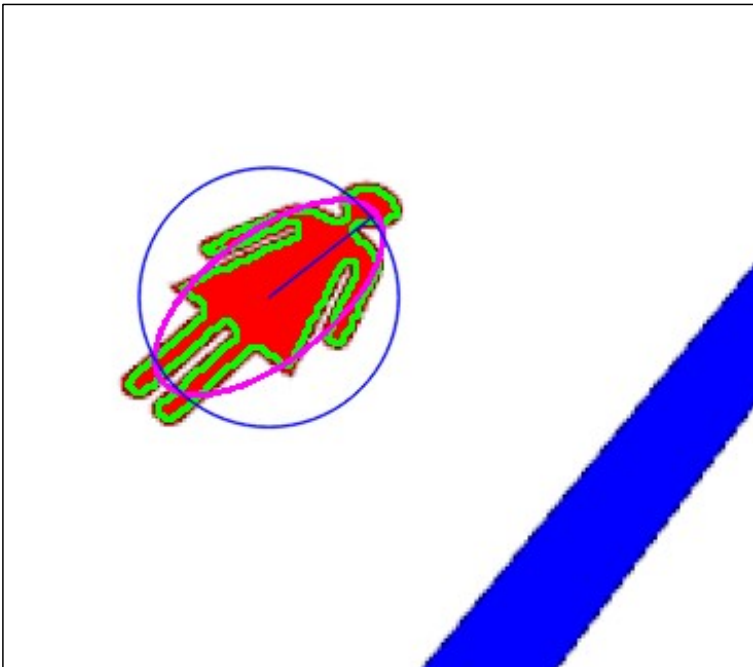
```
lkp, des = orb.compute(labels, [kp])
```

Análisis de imágenes digitales en OpenCV

Implementación OpenCV ORB

Ejemplo descripción de una región:

```
# Pintamos el resultado de la deteccion
cv2.drawContours(imDraw, contList, -1, (0,255,0),2)
cv2.ellipse(imDraw, ellip, (255,0,255),2)
cv2.drawKeypoints(imDraw, lkp, None, color=(0,255,0),
                  flags = cv2.DrawMatchesFlags_DRAW_RICH_KEYPOINTS)
cv2.imshow("Imagen", imDraw), cv2.waitKey(1)
```



Análisis de imágenes digitales en OpenCV

Implementación OpenCV ORB

Ejemplo descripción de una región:

```
# Mostramos el resultado de la descripcion
print('Params Elipse. Centro:', cen, 'Ejes:', ejes, 'Angulo:', angulo)
print('Params ORB. Centro:', cen, 'Escala:', np.mean(ejes)*1.3,
      'Angulo:', angulo-90)
print('Descrip:', des)

Params Elipse. Centro: [516.37628174 321.69781494]
                  Ejes: [ 80.18 126.06]
                  Angulo: 1.490
Params ORB. Centro: [516.37 321.69] Escala: 134.06060066223145
                  Angulo: -88.50
Descrip: [[0 165   0   8 144 129   4 231   6   1  72   0  32  128
          32  0   8  10  2  16  82   8   0  29  40   4   4  96  16
          8 128 136]]

np.unpackbits(des)
array([0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 0, .. .. ..0, 0, 0], dtype=uint8)
```

Análisis de imágenes digitales en OpenCV

Distancia de Hamming

Medida de distancia entre dos strings.

Cuenta el número de posiciones en las que toman valores diferentes

- "karolin" and "kerstin" is 3.
- 1011101 and 1001001 is 2.

```
# Distancia Hamming entre dos descriptores binarios
def hammingDist(d1, d2):
    assert d1.dtype == np.uint8 and d2.dtype == np.uint8
    d1_bits = np.unpackbits(d1)
    d2_bits = np.unpackbits(d2)
    return np.bitwise_xor(d1_bits, d2_bits).sum()
```

Análisis de imágenes de líneas

Tipos de escenas según su complejidad

1. Línea recta



2. Curva a derecha o izquierda



Análisis de imágenes de líneas

Tipos de escenas según su complejidad

3. Cruce / bifurcación 2 salidas



4. Cruce en T con 2 salidas, cruce en X con 3 salidas



Análisis de imágenes de líneas

Tipos de escenas según su complejidad

5. Cruce / incorporación con salida junto a la entrada



Análisis de imágenes de líneas

Asunciones razonables:

1. Las líneas son infinitas (nunca se cortan en medio de la imagen)
2. En una imagen no se pueden ver segmentos futuros o pasados del trazado.
3.

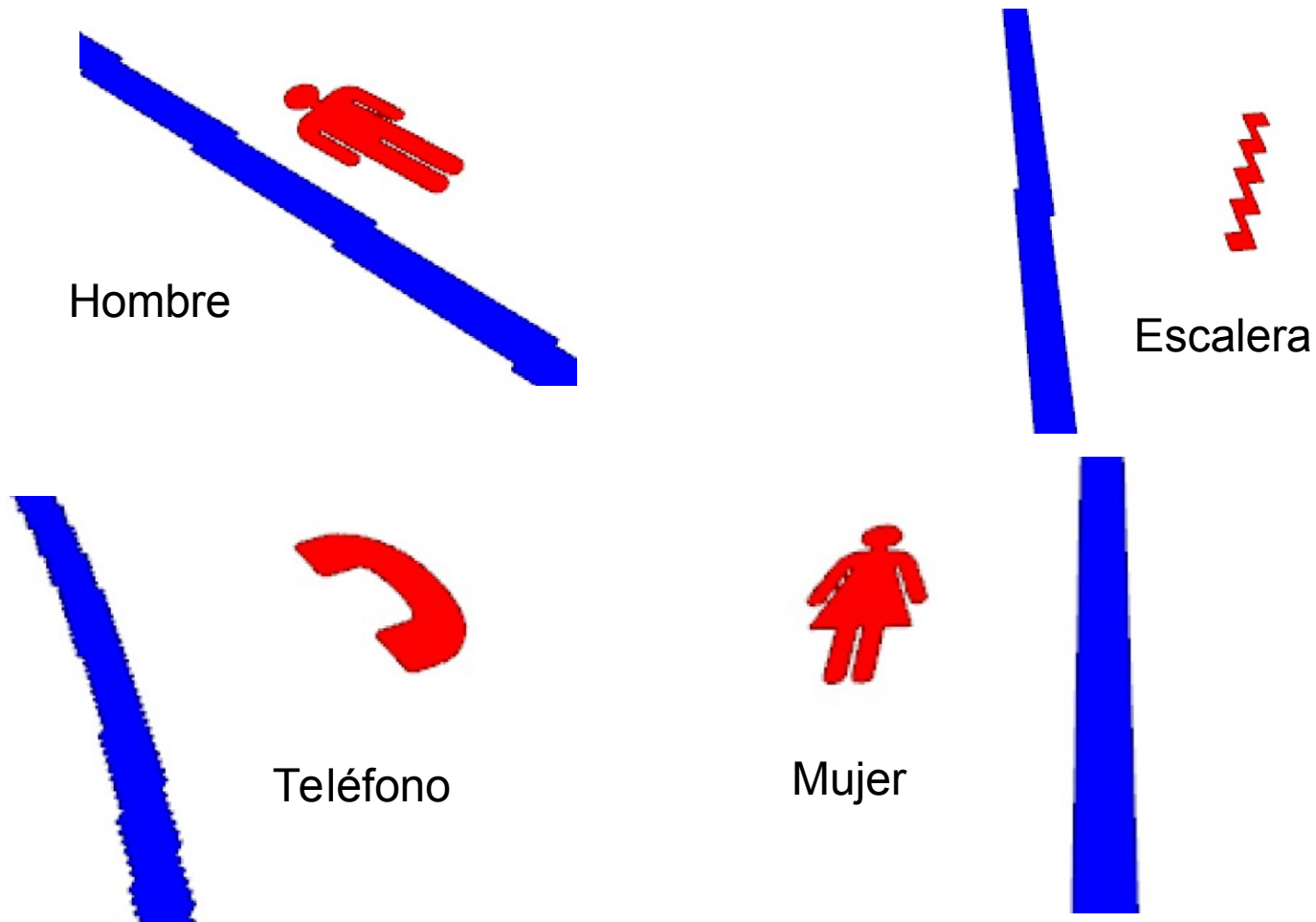
→ Se puede (debe) procesar sólo una parte de la imagen.

Reconocimiento de Marcas

- **Objetivo**

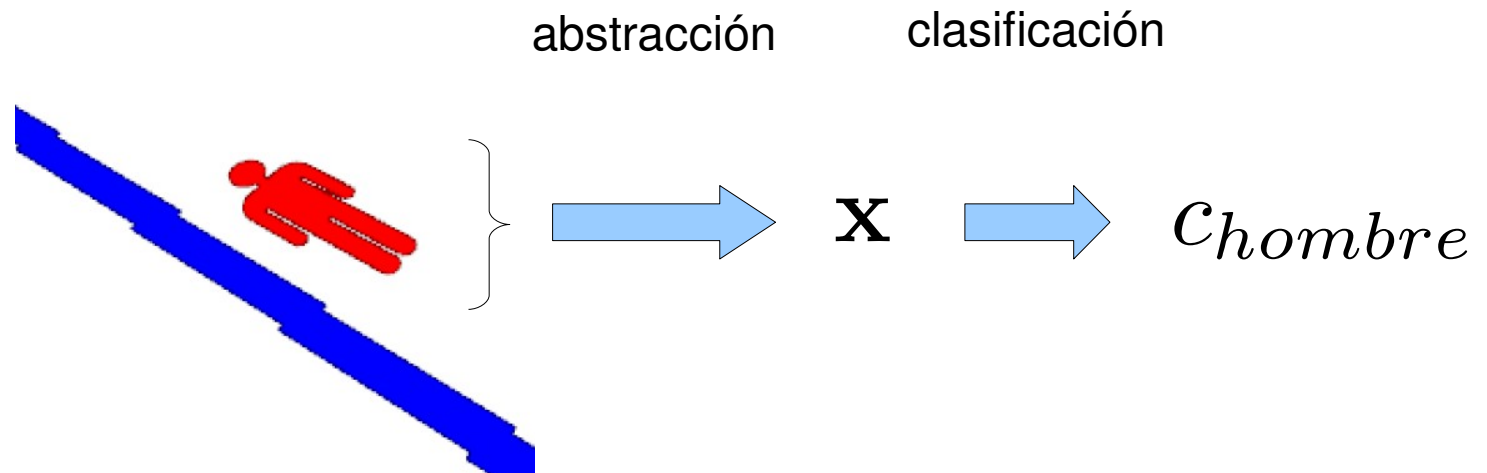
Reconocer 4 tipos de marcas.

Las marcas NO aparecen en cruces, para no confundir con flechas



Modelado del problema

Describimos de forma invariante la marca y le asignamos una etiqueta.



Describimos la marca con un vector de m características discriminantes “invariantes”,

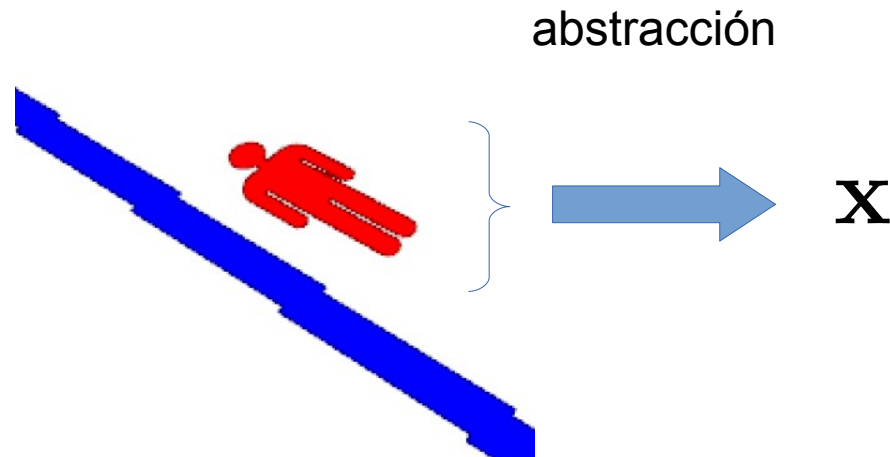
$$\mathbf{x} \in \mathbb{R}^m$$

Representamos las etiquetas mediante un conjunto discreto de valores

$$\{C_{hombre}, C_{mujer}, C_{escalera}, C_{telf}\}$$

Descripción de la marca

Representación $\mathbf{x}$ invariante a traslación, rotación, escalado, deformación proyectiva.



Sugerencia:

Opción 1: Momentos invariantes

Opción 2: Descriptor invariante ORB